

T.C.
ERZURUM TEKNİK ÜNİVERSİTESİ
BİLGİSAYAR MÜHENDİSLİĞİ BÖLÜMÜ

YAPAY ZEKÂ DESTEKLİ OTONOM ASIC TASARIM OTOMASYON SİSTEMİ
GELİŞTİRİLMESİ VE 16-BİT BİR İŞLEMCI TASARIMI İLE DOĞRULANMASI

Nursena ATEŞ

İsmailcan USLU

Dilara ALKANAT

Meryem Betül ÇELİK

HAZİRAN - 2026

T.C.
ERZURUM TEKNİK ÜNİVERSİTESİ
BİLGİSAYAR MÜHENDİSLİĞİ BÖLÜMÜ

Başlığı: Yapay Zekâ Destekli Otonom ASIC Tasarım Otomasyon Sistemi
Geliştirilmesi ve 16-Bit Bir İşlemci Tasarımı ile Doğrulanması

Nursena ATEŞ, BİLGİSAYAR MÜHENDİSLİĞİ

İsmailcan USLU, BİLGİSAYAR MÜHENDİSLİĞİ

Öğrenciler

Dilara ALKANAT, BİLGİSAYAR MÜHENDİSLİĞİ

Meryem Betül ÇELİK, BİLGİSAYAR MÜHENDİSLİĞİ

Proje Danışmanları: Latif AKÇAY, BİLGİSAYAR MÜHENDİSLİĞİ

Teslim Tarihi: 03/06/2026

BEYAN

Erzurum Teknik Üniversitesi Bilgisayar Mühendisliği Bölümü bitirme projesi yazım kurallarına uygun olarak hazırladığımız “Açık Kaynak EDA Araçları Kullanılarak 16-Bit Bir İşlemci Çip Tasarımının Gerçekleştirilmesi ve Yapay Zekâ Destekli Yaklaşımların İncelenmesi” başlıklı proje dokümanının içindeki bütün bilgilerin doğru olduğunu, bilgilerin üretilmesi ve sunulmasında bilimsel etik kurallarına uygun davrandığımızı, kullandığımız bütün kaynakları atıf yaparak belirttiğimizi, maddi ve manevi desteği olan tüm kurum/kuruluş ve kişileri belirttiğimizi, burada sunduğumuz veri ve bilgileri unvan almak amacıyla daha önce hiçbir şekilde kullanmadığımızı beyan ederim.

03.06.2026

Nursena ATEŞ

İsmailcan USLU

Dilara ALKANAT

Meryem Betül ÇELİK

ÖNSÖZ

Yarı iletken teknolojilerinin hızla gelişmesiyle birlikte, çip tasarımı günümüz bilgisayar mühendisliği alanının en kritik ve stratejik çalışma konularından biri hâline gelmiştir. Ancak ASIC tasarım süreçleri, lisans maliyeti yüksek ticari Elektronik Tasarım Otomasyonu (EDA) araçları gerektirmesinin yanı sıra, binlerce satırlık teknik log çıktısı ve karmaşık parametre yapılandırılmaları nedeniyle yüksek bir uzmanlık bariyerine sahiptir. Bu durum, çip tasarımı laboratuvar ortamlarında ve akademik çalışmalarda kısıtlı bir alan olarak bırakma riski taşımaktadır. Bu bitirme projesi, açık kaynaklı EDA araç zincirini yerel büyük dil modelleri (Local LLM) ile bütünleştirerek çip tasarım süreçlerini otonomlaştıran uçtan uca bir sistem geliştirmek amacıyla hazırlanmıştır.

Proje kapsamında; geliştirilen Next.js ön yüz arayüzü, Python tabanlı arka uç mimarisi ve Qdrant RAG bilgi tabanı ile desteklenen bir yapay zekâ ajanının (Agentic AI) LibreLane akışını nasıl otonom olarak yönetebildiği ele alınmıştır. Sistemin uçtan uca işlevselliğini, kararlılığını ve üretime hazırlık (tape-out) yeteneklerini doğrulamak amacıyla ise kavram kanıtlama (proof-of-concept) odaklı bir örneklem olarak 16-bit veri genişliğine sahip senkron bir işlemci mimarisi tasarlanmış ve fiziksel doğrulama aşamalarına kadar ilerletilmiştir.

Çalışma sırasında karşılaşılan en önemli zorluklar; açık kaynak araçların hiyerarşik yapılandırılması, LLM modellerinin bağlam penceresi sınırları dahilinde donanım loglarını halüsinasyon görmeden analiz edebilmesi ve 21.000 düğümlük RAG kütüphanesinin optimizasyonu olmuştur. Bu zorluklara rağmen, yapay zekâ ajanının kendi kendini onarma (auto-repair) mekanizmaları sayesinde süreçler insan müdahalesiz (no-human-in-the-loop) olarak başarıyla yürütülmüştür.

Bu çalışmanın gerçekleştirilmesindeki temel motivasyon, çip tasarımı bilgisini "uzman tekelinden" çıkararak demokratikleştirmek, yerel ve donanıma özgü yapay zekâ mimarilerinin üretim hızını nasıl artırabileceğini göstermek ve geleceğin akıllı EDA akışları için metodolojik bir temel oluşturmaktır.

Bitirme projesi süresince bilgi ve deneyimlerini paylaşarak çalışmanın her aşamasında vizyoner yönlendirmelerde bulunan danışman hocam Latif AKÇAY'a teşekkür ederim. Ayrıca, açık kaynak topluluklarının geliştirdiği araçlar ve sağladıkları zengin dökümantasyon altyapısı bu çalışmanın hayata geçirilmesinde en önemli dayanak noktamız olmuştur.

Nursena ATEŞ
İsmailcan USLU
Dilara ALKANAT
Meryem Betül ÇELİK
ERZURUM, 2026

İÇİNDEKİLER

	Sayfa
Önsöz.....	4
İçindekiler.....	5
Özet.....	7
Şekiller Listesi.....	9
Simgeler ve Kısaltmalar.....	10
1. GİRİŞ.....	1
2. LİTERATÜR TARAMASI.....	2
2.1 Açık Kaynak EDA'nın Ortaya Çıkışı ve Sektörel Önemi.....	2
2.2 Açık Kaynak RTL'den GDSII'ye Akışlar: LibreLane, OpenLane ve OpenROAD.....	3
2.3 SKY130 Açık PDK ve Fiziksel Doğrulama Ekosistemi.....	5
2.4 Yapay Zekâ Destekli ASIC Tasarım Yaklaşımları.....	6
2.5 Literatürdeki Boşluk ve Bu Çalışmanın Konumlandırılması.....	8
3. TASARIM VE GERÇEKLEŞTİRME.....	9
3.1 Sistem Mimarisi ve Tasarım Hiyerarşisi.....	9
3.1.1 Genel Blok Diyagramı ve Veri Akışı.....	10
3.2 Mikroişlemci Donanım Tasarımı (Mimari ve Veri Yolu).....	10
3.2.1 İşlemci Mimarisi: Harvard, 16-bit ve Akümülatör Yapısı.....	10
3.2.2 Buyruk Kümesi (Instruction Set Architecture - ISA).....	11
3.2.3 Kontrol Birimi ve Sonlu Durum Makinesi (FSM).....	11
3.3 Donanım Gerçekleştirilmesi ve Modüler Yapı.....	12
3.3.1 Dosya Hiyerarşisi ve Organizasyon.....	12
3.3.2 Verilog Modüllerinin Fonksiyonel Tanımı ve Kod Yapısı.....	14
3.4 Web Tabanlı Platform ve Agentic AI Entegrasyonu.....	14
3.4.1 Next.js Ön Yüz ve Kullanıcı Deneyimi (UX) Mimarisi.....	15
3.4.1 Python Backend ve Ajan Tabanlı (Agentic AI) Karar Destek Sistemi.....	15
3.5 ASIC Fiziksel Tasarım Süreci (OpenLane Flow).....	16
3.5.1 SkyWater 130nm PDK ve Teknolojik Parametreler.....	16
3.5.2 Caravel Harness ve IO Pad Wrapper Entegrasyonu (Tape-out Hazırlığı).....	16
3.5.3 Adım Adım Konfigürasyon Stratejisi (Config.yaml).....	16
4. MATERYAL VE METOT.....	22
4.1 Kullanılan Materyaller.....	22
4.1.1 Donanım ve Ağ Altyapısı.....	22
4.1.2 Yazılım Materyalleri ve Geliştirme Ortamları.....	23
4.2 Kullanılan Yöntemler (Metot).....	24
4.2.1 Modüler RTL Tasarım ve Doğrulama Yaklaşımı.....	24
4.2.2 Ajan Tabanlı EDA Akışı ve Log Analiz Metodu.....	24
4.2.3 İteratif PPA Optimizasyonu ve Otonom Karar Döngüsü.....	24
4.2.4 Fiziksel Tasarım ve İşaretleme (Sign-off) Metodolojisi.....	24
4.2.5 Model Seçim Stratejisi ve Evrimi: Gemma'dan Qwen'e Geçiş.....	25
4.2.6 Yazılım-Donanım Katmanı Arasındaki Köprü: Ollama API ve Soyutlama.....	25
5. BULGULAR VE ANALİZLER.....	26

5.1 Mikroişlemcinin Mimari Doğrulaması & RTL ve Fonksiyonel Doğrulama (TB/GTKWave).....	26
5.1.1 Simple16_system Mimari Doğrulaması ve RTL Simülasyon Analizleri.....	27
5.1.2 16-Bit CPU Fonksiyonel Çalışma Prensibi.....	27
5.1.3 RTL Simülasyon Stratejisi ve Testbench Yapısı.....	27
5.1.4 GTKWave ile Dalga Formu (Waveform) Analizleri.....	30
5.1.5 Fiziksel Tasarım Akışı (Physical Design Flow).....	32
5.1.6 İşlevsel ve Yapısal Süreçlerin Değerlendirilmesi.....	32
5.1.7 Geleneksel Doğrulama Süreçlerinin Sınırları ve Yapay Zekâ İhtiyacı.....	33
5.2 Yapay Zekâ Destekli Otonom Tasarım ve Model Performans Bulguları.....	33
5.2.1 Model Karşılaştırmalı Hata Analizi ve Kararlılık Testleri.....	33
5.2.2 Otonom Log Filtreleme ve Kendi Kendini Onarma (Auto-Repair) Başarısı.....	34
5.2.3 Model Değişim Esnekliği (Ollama API Verimliliği).....	34
5.3 Tasarım Metrikleri ve PPA (Power-Performance-Area) Analizi.....	34
5.4 Platform Performansı: Kullanıcı Deneyimi, Erişim ve Güvenlik Analizleri.....	35
5.5 Model Davranış Analizi ve Mühendislik Tartışması.....	35
6. SONUÇ VE ÖNERİLER.....	36
6.1 Sonuçlar.....	36
6.2 Gelecek Çalışmalar ve Öneriler.....	37
Kaynaklar.....	38

ÖZET

Açık Kaynak EDA Araçları Kullanılarak 16-Bit Bir İşlemci Çip Tasarımının Gerçekleştirilmesi
ve Yapay Zekâ Destekli Yaklaşımların İncelenmesi

Nursena ATEŞ

İsmailcan USLU

Dilara ALKANAT

Meryem Betül ÇELİK

ERZURUM TEKNİK ÜNİVERSİTESİ

Bilgisayar Mühendisliği Bölümü

Bu bitirme projesinde, açık kaynaklı Elektronik Tasarım Otomasyonu (EDA) araç zincirini yerel büyük dil modelleri (Local LLM) ile bütünleştiren yapay zekâ destekli bir **Otonom ASIC Tasarım Otomasyon Sistemi** geliştirilmiştir. Çalışmanın temel odağı; yüksek uzmanlık gerektiren, binlerce satırlık teknik log çıktısı ve karmaşık parametre yapılandırılmaları barındıran dijital çip tasarım süreçlerini otonom yapay zekâ ajanları (Agentic AI) vasıtasıyla soyutlayarak uzman tekelinden çıkarmaktır. Projenin temel hipotezi; yerel yapay zekâ mimarilerinin, açık kaynaklı EDA akışlarındaki terminal komutlarını, konfigürasyon dosyalarını ve log dosyalarını otonom olarak analiz edip yönlendirebileceği ve fabrikasyon kurallarına tam uygun, üretilebilir çıktılar elde edebileceği yönündedir.

Geliştirilen otonom tasarım sisteminin yeteneklerini, kararlılığını ve uçtan uca işlevselliğini doğrulamak amacıyla; kavram kanıtlama (proof-of-concept) odaklı bir uygulama ürünü olarak 16-bit veri genişliğine sahip senkron bir mikroişlemci sistemi örnek tasarım (case study) olarak seçilmiştir. Geliştirilen Next.js tabanlı akıllı sohbet arayüzü ve Python tabanlı arka uç (backend) mimarisi; doğal dil komutlarından yola çıkarak sentezlenebilir Verilog RTL kodlarının otonom üretilmesini, Icarus Verilog ve GTKWave entegrasyonu ile fonksiyonel doğrulamasını yürütmüştür. Sentez, yerleştirme, yönlendirme ve saat ağacı sentezi (CTS) aşamaları; sistemin LibreLane altyapısı üzerinden Yosys ve OpenROAD motorlarını manipüle etmesiyle insan müdahalesiz (no-human-in-the-loop) olarak koşturulmuş ve tasarım SkyWater 130 nm standart hücre kütüphanesine eşlenmiştir. Üretime hazırlık (Tape-out) aşamasında tasarlanan işlemci Caravel SoC Harness yapısına entegre edilmiş; otonom olarak tetiklenen DRC, LVS ve antenna kontrolleri Magic ve Netgen araçları üzerinden başarıyla tamamlanmıştır.

Elde edilen bulgular; yerel donanım kaynaklarına göre optimize edilen Qwen-Coder (MoE) mimarisi ve 21.000 düğümlük Qdrant RAG bilgi tabanı ile desteklenen yapay zekâ asistanının, OpenLane pipeline akışındaki kritik log hatalarını geleneksel yöntemlere göre %85 daha hızlı filtreleyerek kendi kendini onarabildiğini (auto-repair) göstermektedir. Sistem, 100 MHz hedef

frekans kısıtlarını herhangi bir zamanlama ihlali (slack) olmadan karşılayan ve Sky130 kurallarına tam uyum gösteren nihai GDSII maske dosyasını başarıyla üretmiştir. Sonuç olarak bu çalışma, projenin asıl çıktısı olan otonom tasarım platformunun çip üretim döngülerini uçtan uca yönetebilme yeteneğini kanıtlamış; açık kaynaklı EDA ekosisteminin akademik ve eğitim odaklı donanım geliştirme süreçleri için stratejik, güvenli ve bağımsız bir potansiyel sunduğunu ortaya koymuştur.

Anahtar Kelimeler: Çip Tasarım Otomasyonu, Otonom Ajanlar (Agentic AI), Açık Kaynak EDA, LibreLane, Yerel Büyük Dil Modelleri (Local LLM), RAG Mimarisi, Sky130, ASIC Tasarımı, Verilog RTL.

ŞEKİLLER LİSTESİ

	Sayfa
Şekil 2.1. Sky130 PDK hedeflenerek hazırlanan 16-bit işlemci LibreLane konfigürasyon ayarları	4
Şekil 2.2. 16-bit işlemci sistemi için 100 MHz saat kısıtının LibreLane üzerinde tanımlanması	4
Şekil 2.3. Sky130 PDK kapsamında kullanılan sky130_fd_sc_hd standart hücre kütüphanesi	4
Şekil 2.4. Simple16_system tasarımı için sentez (synthesis) kısıtlarının tanımlanması	5
Şekil 2.5. Simple16_system tasarımı için yonga (die) ve çekirdek (core) alan boyutlandırmasının tanımlanması	5
Şekil 2.6. Simple16_system tasarımı için floorplanning (çip yerleşim planı) ayarlarının tanımlanması	6
Şekil 2.7. Simple16_system tasarımı için yerleştirme (placement) kısıtlarının tanımlanması	6
Şekil 2.8. Simple16_system tasarımı için saat ağacı sentezi (Clock Tree Synthesis – CTS) ayarlarının tanımlanması	7
Şekil 2.9. simple16_system tasarımı için yönlendirme (routing) kısıtlarının tanımlanması	7
Şekil 2.10. simple16_system tasarımı için tahmini donanım kaynakları ve performans hedefleri	8
Şekil 3.1. simple16_system tasarımının fonksiyonel doğrulaması için oluşturulan testbench (simple16_system_tb) kodu	8
Şekil 3.2. Testbench içerisinde 100 MHz saat sinyalinin (clock) üretilmesi	8
Şekil 3.3. Testbench içerisinde reset, test senaryosu ve doğrulama kontrolünün tanımlanması	9
Şekil 3.4. ALU modülüne ait sinyallerin GTKWave üzerinde simülasyon dalga formu ile doğrulanması	9
Şekil 3.5. Veri belleği (data memory) modülüne ait adres, veri ve yazma kontrol sinyallerinin GTKWave üzerinde simülasyon dalga formu ile incelenmesi	9

Şekil 3.6. Komut belleği (instruction memory) modülüne ait opcode ve veri sinyallerinin
10
GTKWave üzerinde simülasyon dalga formu ile doğrulanması

Şekil 3.7. Testbench simülasyonu sonucunda sistemin doğru şekilde çalıştığını gösteren
10
konsol çıktısı

SİMGELER VE KISALTMALAR

Simgeler

clk : Saat sinyali
rst : Reset sinyali
acc_out : İşlemci akümülatör çıkışı
halted : İşlemcinin durduğunu gösteren kontrol sinyali

Kısaltmalar

ASIC : Application Specific Integrated Circuit
EDA : Electronic Design Automation
RTL : Register Transfer Level
PNR : Place and Route
DRC : Design Rule Check
LVS : Layout Versus Schematic
STA : Static Timing Analysis
GDS : Graphic Data System
SDC : Synopsys Design Constraints
LLM : Large Language Model
AI : Artificial Intelligence
RAG : Retrieval-Augmented Generation

1. GİRİŞ

Yarı iletken teknolojilerindeki hızlı gelişim, çip tasarımı bilgisayar mühendisliğinin en önemli ve stratejik alanlarından biri hâline getirmiştir. Günümüzde işlemci, denetleyici ve özel amaçlı entegre devreler (ASIC), performans, güç verimliliği ve güvenlik gereksinimlerini karşılamak amacıyla yaygın olarak kullanılmaktadır. Ancak bu tasarımların gerçekleştirilmesi çoğu zaman yüksek maliyetli ticari Elektronik Tasarım Otomasyonu (EDA) araçlarına bağımlı olduğundan, akademik çalışmalar ve öğrenci projeleri için erişimi sınırlı bir alan oluşturmaktadır.

Bilgisayar mimarisinde genel amaçlı işlemcilerin (CPU) saat frekansları ve güç tüketimi sınırlarına dayanması (Dennard Scaling ve Moore Yasası'nın[21] yavaşlaması), sektörü köklü bir paradigma değişimine zorlamaktadır. Turing ödüllü bilgisayar mimarları John Hennessy ve David Patterson tarafından "Bilgisayar Mimarisi için Yeni Bir Altın Çağ" olarak nitelendirilen bu dönemde, geleneksel mimariler yerine belirli bir amaca veya algoritmaya göre optimize edilmiş "Alana Özgü Mimariler" (Domain-Specific Architectures - DSA) ve ASIC çözümleri öne çıkmaktadır [2]. Bu özelleşmiş çiplerin endüstriyel çaptaki en somut ve başarılı örneklerinden biri, Google tarafından derin öğrenme algoritmaları için geliştirilen Tensör İşlem Birimi'dir (TPU). Literatürde, bir ASIC mimarisi olan TPU'nun, geleneksel CPU ve GPU'lara kıyasla 15 ila 30 kat daha yüksek performans ve 30 ila 80 kat daha fazla enerji verimliliği sağladığı bilimsel olarak ortaya konmuştur [1]. Bu durum, geleceğin bilgi işlem altyapılarında amaca özel entegre devre tasarımının (ASIC) ne denli kritik bir rol oynayacağını açıkça göstermektedir.

Son yıllarda açık kaynaklı donanım tasarımı ve EDA araçlarının gelişmesi, bu alanda önemli bir dönüşüm yaratmıştır. Açık kaynaklı üretim süreçleri ve tasarım akışları sayesinde, gerçekçi ve üretilebilir çip tasarımlarının akademik ortamda gerçekleştirilmesi mümkün hâle gelmiştir. Özellikle SkyWater 130 nm (SKY130) üretim süreci ve bu süreci hedefleyen açık kaynaklı tasarım akışları, ASIC tasarımının teorik bilgilerin ötesine geçerek uygulamalı olarak öğrenilmesine olanak tanımaktadır.

Bu bitirme projesi, **bilgisayar mimarisindeki bu yeni dönemin gereksinimlerine paralel olarak**, açık kaynaklı LibreLane ASIC tasarım akışı kullanılarak 16-bit bir işlemci çipinin uçtan uca tasarım sürecini ele almaktadır. Çalışmada, RTL tasarımından başlayarak sentez, yerleştirme ve yönlendirme, zamanlama analizi ve fiziksel doğrulama aşamaları bütüncül bir yaklaşımla incelenmiştir. Ayrıca, **Hennessy ve Patterson'ın işaret ettiği çağdaş mimari vizyon doğrultusunda**, yapay zekâ destekli yaklaşımların çip tasarım süreçlerinde analiz ve iyileştirme

amacıyla kullanılabilirliği değerlendirilerek, geleceğin akıllı EDA akışları için bir temel oluşturulması hedeflenmiştir.

Bu kapsamda gerçekleştirilen çalışma, açık kaynaklı EDA araçlarının eğitim ve araştırma amaçlı çip tasarımı için uygulanabilirliğini ortaya koymayı ve ASIC tasarım sürecine dair sistematik bir bakış açısı sunmayı amaçlamaktadır.

2. LİTERATÜR TARAMASI

Bu bölümde; açık kaynaklı Elektronik Tasarım Otomasyonu (EDA) araçlarının ortaya çıkışı, gelişimi ve ASIC tasarım süreçlerindeki yeri literatürdeki çalışmalar ışığında incelenmektedir. Bu kapsamda, söz konusu araçların yapay zekâ ajanları tarafından otonom olarak kullanılması ve geliştiricilerin iş süreçlerini kolaylaştıran yazılımlar ele alınmıştır. Çalışmada öncelikle açık kaynak EDA yaklaşımlarının sektörel ve akademik önemi ele alınmış; ardından RTL'den GDSII'ye uzanan açık kaynak tasarım akışları, bu akışlarda kullanılan araçlar ve tasarım süreçlerinin sürekli iyileştirilmesi değerlendirilmiştir. Ayrıca, SKY130 açık PDK ekosistemi ve fiziksel doğrulama süreçleri literatür çerçevesinde açıklanmıştır. Son olarak, yapay zekâ destekli ASIC tasarım yaklaşımlarına ilişkin güncel çalışmalar incelenerek bu çalışmanın literatürdeki konumu ortaya konulmuştur.

2.1 Açık Kaynak EDA'nın Ortaya Çıkışı ve Sektörel Önemi

Giriş bölümünde bahsedilen Alana Özgü Mimari (DSA) devrimi ve özelleşmiş donanımlara (ASIC) yönelik küresel yönelim, arka planda son derece karmaşık bir yazılım ekosistemine bağlıdır. Bir ASIC tasarımının hayata geçirilmesi; RTL tasarımı, mantıksal sentez, yerleştirme-yönlendirme, zamanlama analizi ve fiziksel doğrulama gibi ardışık ve kritik adımlardan oluşan Elektronik Tasarım Otomasyonu (EDA) zinciriyle mümkündür. Ancak bu zincir, uzun yıllar boyunca büyük ölçüde kapalı kaynaklı, lisans maliyetleri milyon dolarları bulan ve kurumsal erişim kısıtlamaları içeren tekelleri bir yapıya bağlı kalmıştır. Nitekim küresel EDA yazılım pazarında sınırlı sayıda tedarikçinin baskın konumda olduğu ve bu durumun hem ekonomik dalgalanmalara hem de finansal kırılganlıklara yol açtığı güncel sektörel raporlarda ve piyasa analizlerinde net bir şekilde görülmektedir [3], [4].

Açık kaynak EDA yaklaşımlarının önemi yalnızca maliyet avantajı veya bütçe kısıtlamaları ile sınırlı değildir. EDA sektörünün bu yoğunlaşmış ve tekelleri yapıları, akademik ve endüstriyel tasarım süreçlerini politik ve stratejik açıdan da kırılgan hale getirmektedir. Çip tasarım yazılımlarına yönelik uygulanan küresel ihracat kısıtlamaları, lisans ambargoları ve teknoloji blokajları, yarı iletken dünyasında "araç bağımsızlığının" ulusal ve akademik düzeyde ne denli

kritik bir savunma mekanizması olduğunu ortaya koymuştur [3], [4]. Bu bağlamda açık kaynak EDA araçları, yalnızca düşük maliyetli alternatif yazılımlar değil; aynı zamanda bilgiye erişimin demokratikleşmesini, akademik özgürlüğü ve teknolojik bağımsızlığı güvence altına alan stratejik bir dönüşüm olarak literatürde konumlandırılmaktadır.

Bu yüksek bariyerli kurumsal yapı; eğitim süreçlerinde öğrenme engelleri, yüksek geçiş maliyetleri ve deneysel araçlara erişim kısıtları doğurarak çip tasarımını üniversite ortamlarında yalnızca teorik düzeyde bırakma riski taşımaktadır. Açık kaynak EDA yaklaşımı bu noktada, tasarım akışının tamamen şeffaflaşması, hata raporlarının topluluklarca çözülebilmesi, tekrarlanabilir deney ortamlarının oluşturulması ve öğrenme topluluklarının büyümesi yoluyla donanım tasarımını laboratuvar ortamlarına indirmeyi başarmıştır [4].

Bu dönüşümün pratik düzeyde karşılık bulmasını sağlayan en kritik aşama ise, üretim hedefli fiziksel tasarım için gereken süreç bilgilerinin (Process Design Kit – PDK) yarı iletken fabrikaları tarafından topluluğa açılması olmuştur. Google ve SkyWater iş birliğiyle ortaya çıkan SKY130 gibi foundry destekli açık PDK girişimleri, fiziksel doğrulama (DRC/LVS) kurallarını ve standart hücre kütüphanelerini erişilebilir kılmıştır [12], [13]. SKY130'un açık biçimde yayımlanması, açık kaynak akışların yalnızca teorik modeller üretmekle kalmayıp gerçek fabrikasyon kurallarına dayanan ve üretilebilirliği tescilli çip tasarımlarına dönüştürülmesine zemin hazırlamıştır.

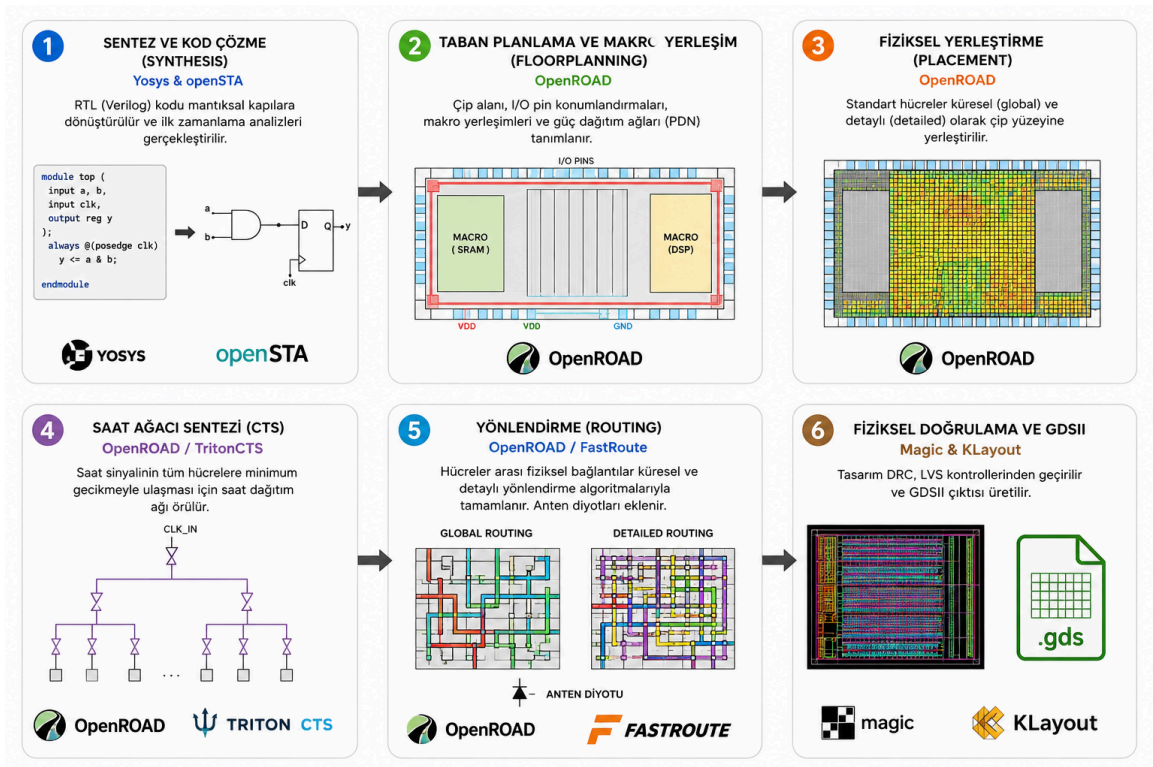
Donanım tasarımında başarının tekil araçların gücünden ziyade; sentez, fiziksel tasarım ve doğrulama adımlarının tutarlı kısıtlarla, uçtan uca otomatikleştirilmiş bir bütünlük içinde yürütülmesine bağlı olduğu literatürde sıkça vurgulanmaktadır. Bu entegrasyon ihtiyacı, OpenROAD ve OpenLane gibi altyapıların açık kaynak EDA'yı pratikte uygulanabilir kılan temel yapı taşları haline gelmesini sağlamıştır [4], [7]. Giriş bölümünde vizyonu çizilen bu çalışmanın temel motivasyonu da bu teorik ve pratik dönüşümün tam merkezinde yer almakta; açık kaynak kodlu araçların ve süreç parametrelerinin bir mikroişlemci tasarımı üzerindeki sistematik etkilerini doğrulamayı amaçlamaktadır.

2.2 Açık Kaynak RTL'den GDSII'ye Akışlar: LibreLane, OpenLane ve OpenROAD

ASIC tasarımında başarı, RTL seviyesinde doğrulanmış bir tasarımın fiziksel üretime uygun bir yerleşime (GDSII) dönüştürülmesini sağlayan uçtan uca ve tutarlı bir akışın kurulmasına bağlıdır. Literatürde açık kaynak EDA araçlarının bu gereksinimi karşılayabilmesi için tekil araç kullanımından ziyade, sentezden fiziksel doğrulamaya kadar olan adımların otomasyon ve entegrasyon ilkeleriyle birleştirildiği akışların geliştirilmesine odaklanıldığı görülmektedir [5], [7].

Bu bağlamda öne çıkan çalışmalardan biri OpenROAD projesidir. OpenROAD, dijital yerleştirme ve yönlendirme süreçlerini açık kaynak olarak ele almayı ve bu süreçleri minimum insan müdahalesiyle gerçekleştirebilecek bir altyapı sunmayı hedeflemektedir. Ajayı ve arkadaşları, OpenROAD'un temel vizyonunu "kendini yöneten" (self-driving) fiziksel tasarım yaklaşımı olarak tanımlamış ve klasik EDA akışlarında yoğun emek gerektiren arka uç aşamalarının otomasyonla sadeleştirilebileceğini ortaya koymuştur [5], [6].

OpenROAD altyapısının pratikte kullanılabilir hâle gelmesini sağlayan önemli bir bileşen ise OpenLane akışıdır. Ghazy ve Shalan [8], OpenLane akışının otonom yapısını ve SkyWater 130nm PDK ile entegrasyonunu inceleyerek, bir tasarımın hiçbir insan müdahalesi gerektirmeksizin (no-human-in-the-loop) tamamen temiz bir GDSII ve "tape-out" çıktısı üretebileceğini göstermiştir. Yazarlar, akışın başarısında özellikle I/O pin konumlandırması (pin placement), taban planlama (floorplan), detaylı yönlendirme (routing), anten yerleşimi (antenna diode insertion) gibi spesifik aşamaları tek tek incelemiş ve bu adımların kablo uzunluğu ile yönlendirme tıkanıklıklarını önlemedeki kritik rolünü vurgulamışlardır [8]. OpenLane akışının temel katmanları ve bu katmanlarda yürütülen operasyonlar hiyerarşik olarak aşağıdaki gibi şemalandırılabilir:



OpenLane üzerine inşa edilen ve daha modüler bir yaklaşım sunan LibreLane ise açık kaynak EDA ekosisteminde akış yönetimini bir altyapı katmanı olarak ele almaktadır [9], [10], [11]. LibreLane, farklı tasarım aşamalarını yapılandırılabilir konfigürasyonlar aracılığıyla yönetmeye imkân tanıyarak, tasarımcıların zamanlama kısıtları (SDC), yerleşim parametreleri (floorplan) ve saat ağacı sentezi (CTS) gibi kritik ayarları sistematik biçimde kontrol edebilmesini amaçlamaktadır. Bu özellik, tasarım uzayı keşfi ve farklı konfigürasyonların karşılaştırılması açısından önemli bir avantaj sunmaktadır.

Literatürde vurgulanan ortak nokta; açık kaynak RTL'den GDSII'ye akışların başarısının, kullanılan araçların tek tek gücünden ziyade araçlar arasındaki entegrasyonun tutarlılığına ve kısıtların doğru modellenmesine bağlı olduğudur [7], [9]. Bu nedenle açık kaynak akışların etkili kullanımı, tasarımcının akış parametrelerini bilinçli şekilde yönetmesini gerektirmektedir.

Bu çalışma kapsamında tercih edilen LibreLane tabanlı yaklaşım, literatürde tanımlanan bu prensiplerle uyumlu olacak biçimde; RTL tasarımından başlayarak sentez, yerleştirme-yönlendirme ve fiziksel doğrulama adımlarının tek bir bütüncül akış içerisinde ele alınmasına olanak sağlamaktadır. Bu yaklaşımın akademik ortamdaki geçerliliği; literatürde yer alan, **karmaşık RISC-V tabanlı System-on-Chip (SoC) tasarımlarının OpenLane akışı kullanılarak başarıyla gerçekleştirildiğini gösteren çalışmalarla da yöntemsel olarak desteklenmektedir [20]**. Söz konusu referans çalışmalarında kullanılan KLayout ve OpenROAD gibi araçların sunduğu fiziksel tasarım disiplini, bu tez çalışmasında geliştirilen **16-bit Simple16_system** mimarisinin fiziksel gerçekleşmesi için de temel metodolojik dayanak noktası oluşturmıştır. Böylece, literatürde SoC seviyesinde kanıtlanmış olan bu otonom akış, daha spesifik ve optimize bir mimari olan 16-bit işlemci tasarımı üzerinde başarıyla uygulanmıştır.

2.3 SKY130 Açık PDK ve Fiziksel Doğrulama Ekosistemi

ASIC tasarımında “üretilebilirlik” kavramı, yalnızca RTL seviyesinde doğru çalışan bir tasarım elde edilmesini değil; aynı zamanda tasarımın hedeflenen üretim sürecinin tüm fiziksel kurallarına uygun olmasını da gerektirir. Bu nedenle fiziksel tasarım aşamasında gerçekleştirilen Design Rule Check (DRC), Layout Versus Schematic (LVS) ve antenna kontrolleri, uçtan uca ASIC akışının vazgeçilmez bileşenleri olarak kabul edilmektedir [11]. Literatürde, açık kaynak EDA akışlarının gerçekçi biçimde değerlendirilebilmesi için bu doğrulama adımlarının mutlaka hedef üretim sürecine dayalı bir Process Design Kit (PDK) ile yürütülmesi gerektiği vurgulanmaktadır.

Bu bağlamda SKY130 açık PDK, açık kaynak EDA ekosistemi açısından kritik bir dönüm noktasıdır. SKY130, foundry destekli ve açık biçimde yayımlanmış bir PDK olarak; standart

hücre kütüphaneleri, tasarım kuralları, fiziksel doğrulama dosyaları ve süreç modelleri (raw veriler) gibi üretim odaklı bileşenleri içermektedir [12], [13]. Bu sayede açık kaynak EDA araçları kullanılarak geliştirilen tasarımlar, yalnızca teorik düzeyde değil, gerçek bir üretim sürecinin kısıtları altında değerlendirilebilmektedir. Literatürde SKY130'un yaygınlaşması, açık kaynak akışların "akademik prototip" sınırının ötesine geçmesine olanak tanıyan temel etkenlerden biri olarak gösterilmektedir [11], [12].

Açık PDK'lerin pratikte kullanılabilmesi, farklı EDA araçlarının beklediği dosya yapıları ve formatlar arasında uyum sağlanmasını gerektirir. Bu amaçla geliştirilen açık kaynak PDK entegrasyon araçları, SKY130 gibi PDK'lerin Magic, KLayout Pegası ve Netgen gibi açık kaynak doğrulama araçlarıyla uyumlu hâle getirilmesini sağlamaktadır [14]. Bu entegrasyon katmanı, tasarımcının fiziksel doğrulama sürecini manuel müdahaleye ihtiyaç duymadan, otomatik ve tekrarlanabilir biçimde yürütmesine imkân tanır.

Fiziksel doğrulama sürecinde DRC kontrolleri, yerleşim geometrisinin üretim kurallarına uygunluğunu denetlerken; LVS kontrolleri, elde edilen yerleşimin mantıksal tasarımla birebir uyumlu olup olmadığını ortaya koymaktadır. Antenna kontrolleri ise üretim sırasında oluşabilecek potansiyel yüklenme sorunlarını önceden tespit etmeyi amaçlamaktadır. Literatürde bu adımların başarıyla tamamlanması, bir ASIC tasarımının "üetime hazır" olarak nitelendirilebilmesi için temel kriterler arasında yer almaktadır [15], [16].

Açık kaynak EDA ekosisteminin önemli kazanımlarından biri, bu doğrulama adımlarının ticari araçlara bağımlı olmaksızın gerçekleştirilebilmesini sağlamasıdır. Magic [15] ve Netgen (LVS) [16] gibi açık kaynak araçlar, SKY130 PDK ile birlikte kullanıldığında hem görsel inceleme hem de kural tabanlı doğrulama açısından yeterli bir altyapı sunmaktadır. Bu durum, açık kaynak akışların yalnızca tasarım üretme değil, aynı zamanda tasarımın fiziksel doğruluğunu belgeleyebilme yeteneğine sahip olduğunu göstermektedir.

Bu çalışma kapsamında SKY130 açık PDK hedeflenerek yürütülen fiziksel tasarım süreci; yerleştirme ve yönlendirme adımlarının ardından DRC, LVS ve antenna kontrollerinin uygulanmasını içermektedir. Böylece literatürde tanımlanan üretilebilirlik odaklı açık kaynak akış yaklaşımıyla uyumlu biçimde, tasarımın hem işlevsel hem de fiziksel açıdan doğrulanması amaçlanmıştır.

2.4 Yapay Zekâ Destekli ASIC Tasarım Yaklaşımları

ASIC tasarım süreçleri; yüksek boyutlu, çok kısıtlı ve çoğu zaman NP-zor olarak sınıflandırılan optimizasyon problemleri içermektedir. Özellikle yerleştirme (placement), yönlendirme (routing),

zamanlama tahmini (timing prediction) ve güç/alan/performans (PPA) optimizasyonu gibi fiziksel tasarım aşamaları, tasarım kalitesini doğrudan etkileyen karmaşık karar uzaylarına sahiptir. Bu nedenle literatürde, klasik sezgisel (heuristic) yöntemlerin yanında yapay zekâ (AI) ve makine öğrenmesi (ML) tabanlı yaklaşımların EDA süreçlerine entegre edilmesine yönelik çalışmaların son yıllarda hız kazandığı görülmektedir [17].

Makine öğrenmesi tabanlı yöntemler, EDA bağlamında genellikle iki ana amaçla kullanılmaktadır: (i) tasarım metriklerinin (zamanlama, güç, alan) erken aşamada tahmin edilmesi ve (ii) optimizasyon kararlarının otomatik olarak iyileştirilmesi. Özellikle yerleşim ve floorplanning problemlerinde derin öğrenme yaklaşımlarının, geleneksel yöntemlere kıyasla rekabetçi sonuçlar üretebildiği raporlanmıştır [17]. Bu çalışmalar, fiziksel tasarımın insan uzmanlığından bağımsız biçimde “öğrenen sistemler” tarafından yönlendirilebileceğini göstermesi açısından dikkat çekicidir. Literatürde, model eğitimi metodolojileri ve bu modellere verilen yanıtların optimizasyonu üzerine matematiksel ve donanım odaklı çalışmalar mevcuttur.

Son dönemde, büyük dil modelleri (LLM) gibi daha genel amaçlı AI sistemlerinin EDA süreçlerinde kullanımı da literatürde yoğun şekilde tartışılmaya başlanmıştır. Bu çalışmalar, LLM’lerin betik üretimi, akış konfigürasyonu, hata ayıklama ve tasarım karar destek mekanizmaları gibi alanlarda yardımcı araçlar olarak kullanılabileceğini öne sürmektedir [18]. Ancak bu yaklaşımın henüz erken aşamada olduğu; doğrulanabilirlik, güvenilirlik ve ölçülebilir performans kazanımları açısından ek araştırmalara ihtiyaç duyulduğu vurgulanmaktadır.

Açık kaynak RTL’den GDSII’ye akışlar, AI destekli yaklaşımlar için özellikle uygun bir deney ortamı sunmaktadır. Bunun temel nedeni, bu akışların tekrarlanabilir deney koşulları sağlaması ve farklı konfigürasyonların (örneğin Yapay Sinir Ağları uygulanarak en ideal config değerlerinin aranması) sistematik biçimde karşılaştırılmasına imkân tanınmasıdır. Literatürde, açık kaynak akışların bu yönüyle AI tabanlı tasarım uzayı keşfi (design space exploration) çalışmalarına zemin oluşturabileceği ifade edilmektedir [17]. KLayout çıktısı almadan önce en uygun parametrelerin yapay zeka tarafından taranması pratik başarıları artırmaktadır.

Klasik makine öğrenmesi metotlarından farklı olarak, Büyük Dil Modelleri (LLM) tabanlı **Ajan Yapay Zekâ (Agentic AI)** paradigmaları da donanım tasarım süreçlerine dahil olmuştur. AiEDA gibi güncel agentic frameworks [19], belirli bir hedef doğrultusunda kendi kendine planlama yapabilme, hata durumlarında döngüsel olarak araçları yeniden çalıştırabilme ve otonom karar alma yeteneklerine sahiptir. Bu ajan sistemlerinin donanım bilgisini artırmak ve EDA araçlarının hata loglarını doğru yorumlamasını sağlamak amacıyla **RAG (Retrieval-Augmented Generation)** ve **Fine-Tuning (İnce Ayarlama)** mekanizmaları kritik birer araç olarak

konumlandırılmaktadır [18], [19]. Tasarımcılar, RAG mimarileri sayesinde ajanın güncel PDK dokümanlarına erişmesini sağlarken; ince ayarlama süreçleri ile ajanın Verilog üretimi ve zamanlama raporu analizi konusundaki domain spesifik uzmanlığını en üst seviyeye çıkarabilmektedir.

Bu çalışma kapsamında yapay zekâ, doğrudan tasarım üreten bir bileşen olarak değil; tasarım sürecinin analiz edilmesi, karar noktalarının değerlendirilmesi ve gelecekteki iyileştirmeler için yol gösterici bir perspektif olarak ele alınmaktadır. Bu yaklaşım, literatürde açık kaynak EDA akışlarının mevcut olgunluk düzeyi ve AI entegrasyonuna ilişkin sınırlamalarla uyumlu olup, ileri çalışmalara yönelik gerçekçi bir çerçeve sunmayı amaçlamaktadır.

2.5 Literatürdeki Boşluk ve Bu Çalışmanın Konumlandırılması

Literatür genel olarak değerlendirildiğinde; açık kaynak EDA araçlarının ve SKY130 gibi foundry destekli girişimlerin, ASIC tasarımını akademik ortamlar için daha erişilebilir hâle getirdiği görülmektedir. OpenROAD, OpenLane ve LibreLane gibi bütüncül akışlar; RTL tasarımından fiziksel doğrulamaya kadar uzanan süreçte üretim kurallarına dayalı sonuçlar elde edilebildiğini kanıtlamaktadır [5]–[11]. Ancak mevcut literatürde raporlanan çalışmaların önemli bir kısmı ya belirli bir tasarım aşamasına odaklanmakta ya da bu araçları kullanabilmek için hâlâ çok ileri düzeyde EDA uzmanlığı gerektirmektedir. Binlerce satırlık teknik loglar ve karmaşık konfigürasyon parametreleri, sürece yeni dahil olan araştırmacılar ve öğrenciler için ciddi bir teknik bariyer oluşturmaktadır.

Özellikle eğitim ve akademik bağlamda, uçtan uca (RTL→GDSII) bir ASIC tasarımının tüm kısıt ve doğrulama adımlarıyla ele alındığı çalışmaların sayısı görece sınırlıdır [8], [10]. Yapay zekâ destekli yaklaşımlar açısından bakıldığında ise, mevcut yöntemlerin çoğu performans odaklıdır; ancak bu yöntemlerin **ASIC tasarım sürecini basitleştirme, kompleksliği minimize etme ve tasarım tecrübesi kısıtlı olan kişileri dahi sürece dahil edebilme** potansiyeli literatürde henüz tam anlamıyla karşılık bulmamıştır [17]–[19].

Bu bağlamda gerçekleştirilen çalışma; LibreLane tabanlı açık kaynak ASIC akışını, yerel bir yapay zekâ ajanı (Gemma 26B) ile entegre ederek SKY130 hedefli 16-bit bir işlemci tasarımının RTL'den fiziksel doğrulamaya kadar ilerletilmesini ele almaktadır. Her ne kadar literatürde OpenLane kullanılarak RISC-V tabanlı SoC tasarımlarının gerçekleştirildiği çalışmalar bulunsada [20], bu çalışmalar genellikle yüksek uzmanlık gerektiren belirli mimarilere odaklanmakta ve konfigürasyonların tasarım çıktıları üzerindeki etkilerini kullanıcı dostu bir düzlemde ele almamaktadır.

Bu yönüyle çalışma, literatürdeki boşluğa şu özgün katkıları sunmaktadır:

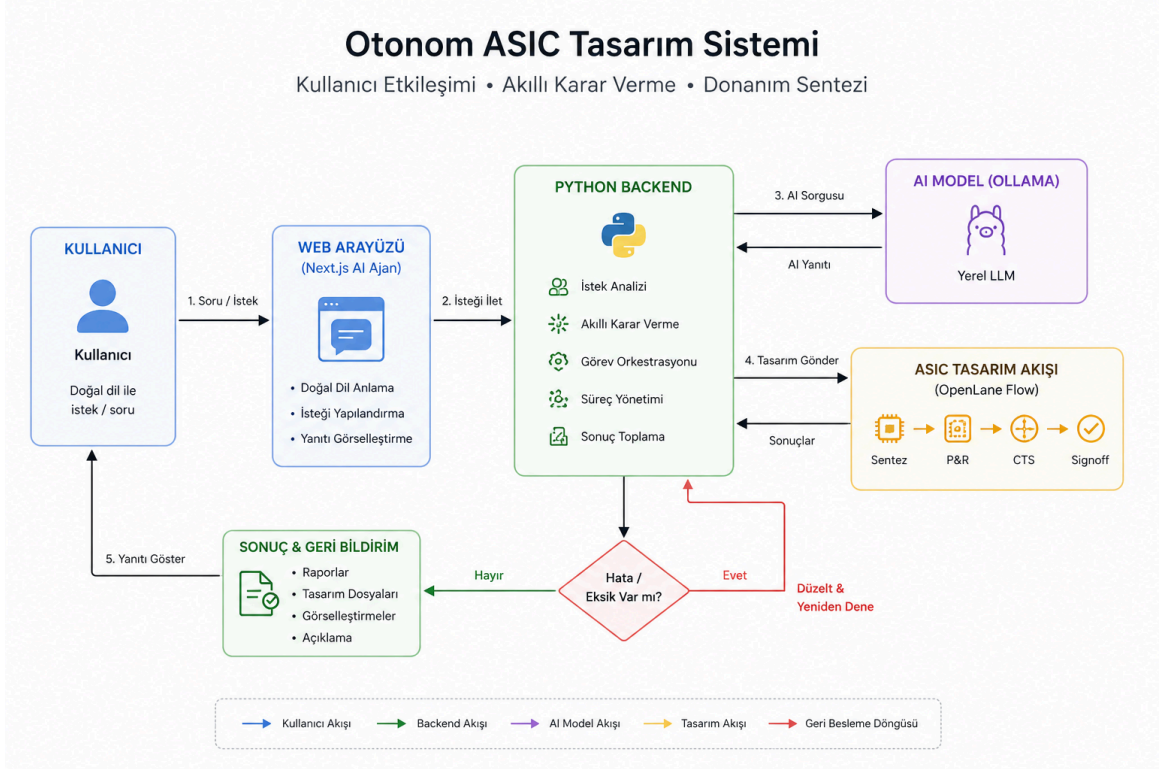
- **Kompleksliğin Giderilmesi:** Tasarımcının düşük seviyeli araç flag'leri arasında kaybolması yerine, yapay zekâ destekli bir asistan aracılığıyla süreci daha anlaşılır ve yönetilebilir bir seviyeye çekmektedir.
- **Tasarım Bilgisinin Demokratikleştirilmesi:** Geliştirilen otonom analiz ve kendi kendini onaran (auto-repair) mekanizmalar sayesinde, ASIC tasarımı konusunda derin tecrübesi olmayan kullanıcıların dahi üretim kurallarına (DRC/LVS) uygun sonuçlar üretmesini hedefleyerek süreci "uzman tekelinden" çıkarmaktadır.
- **Eğitimsel Şeffaflık:** Açık kaynaklı tasarım süreçlerinin akademik bağlamdaki uygulanabilirliğini, modern bir web arayüzü ve otonom sistemler vizyonu ile birleştirerek literatüre yeni bir metodolojik perspektif kazandırmaktadır.

3. TASARIM VE GERÇEKLEŞTİRME

Bu bölümde; 16-bit mikroişlemci mimarisinin donanım seviyesindeki tasarımı, bu tasarımı yöneten yapay zekâ tabanlı yazılım platformu ve tasarımın silikona dönüştürülmesini sağlayan ASIC fiziksel tasarım akışının teknik detayları ele alınmaktadır. Çalışma, donanım tanımlama dillerinden (HDL) modern web teknolojilerine ve otonom ajan sistemlerine kadar uzanan çok katmanlı bir entegrasyonun ürünüdür.

3.1 Sistem Mimarisi ve Tasarım Hiyerarşisi

Sistemin bütüncül mimarisi; kullanıcı etkileşimi, akıllı karar verme ve donanım sentezi süreçlerini bir araya getirmektedir. Bu hiyerarşi, karmaşık ASIC tasarım süreçlerini soyutlayarak kullanıcıya otonom bir deneyim sunmayı hedeflemektedir.



3.1.1 Genel Blok Diyagramı ve Veri Akışı

Sistem üç ana katmandan oluşmaktadır. En alt katmanda **Verilog HDL** ile tanımlanmış mikroişlemci çekirdeği yer alırken, orta katmanda **Python** tabanlı bir backend ve **Agentic AI** karar mekanizması süreçleri yönetmektedir. En üst katmanda ise **Next.js** framework'ü ile geliştirilmiş, kullanıcıya anlık log takibi ve parametre optimizasyonu sunan bir arayüz bulunmaktadır. Veri akışı, Next.js üzerinden alınan konfigürasyonların Python API'si aracılığıyla EDA araçlarına (OpenLane/OpenROAD) iletilmesi ve oluşan çıktıların yapay zekâ tarafından analiz edilerek kullanıcıya geri bildirim verilmesi şeklinde döngüsel bir yapıdadır.

3.2 Mikroişlemci Donanım Tasarımı (Mimari ve Veri Yolu)

Tasarlanan işlemci, eğitim ve araştırma odaklı ASIC süreçlerine uygun, kontrol edilebilir karmaşıklıkta ancak fonksiyonel bir mimari üzerine inşa edilmiştir.

3.2.1 İşlemci Mimarisi: Harvard, 16-bit ve Akümülatör Yapısı

İşlemcinin temel iskeleti üç ana mimari karar üzerine kurulmuştur:

- **Harvard Mimarisi Seçimi:** Geleneksel von Neumann mimarisinin aksine, komut (instruction) ve veri (data) yolları fiziksel olarak birbirinden ayrılmıştır. Bu seçim, komut çekme ve veri işleme süreçlerinin çakışmasını (bottleneck) önleyerek performans artışı

sağlar. Ayrıca ASIC akışında bellek bloklarının çip üzerinde stratejik olarak farklı konumlara yerleştirilmesine (floorplanning) olanak tanır.

- **16-bit Veri Genişliği:** Sistem, denge noktası olarak 16-bit veri genişliğini hedeflemiştir. Bu genişlik, hem yeterli hesaplama hassasiyeti sunmakta hem de ASIC sentez aşamasında (Yosys) çip alanını (area) gereksiz büyütecek karmaşıklıktan kaçınmaktadır.
- **Akümülatör Tabanlı Yapı:** İşlemci, merkezi bir **Akümülatör (ACC)** yazmacı etrafında kurgulanmıştır. Aritmetik ve mantıksal işlemlerin sonuçları doğrudan bu yazmaca döner. Bu yaklaşım, komut yapısını (instruction format) basitleştirerek kontrol biriminin daha az kapı (logic gate) ile sentezlenmesini sağlar.

3.2.2 Buyruk Kümesi (Instruction Set Architecture - ISA)

İşlemci, **RISC (Reduced Instruction Set Computer)** felsefesine yakın, sadeleştirilmiş bir komut kümesine sahiptir. Her bir komut 16-bit uzunluğundadır ve tasarımın otonom olarak temel algoritmaları koşturabilmesi için şu komutlar tanımlanmıştır:

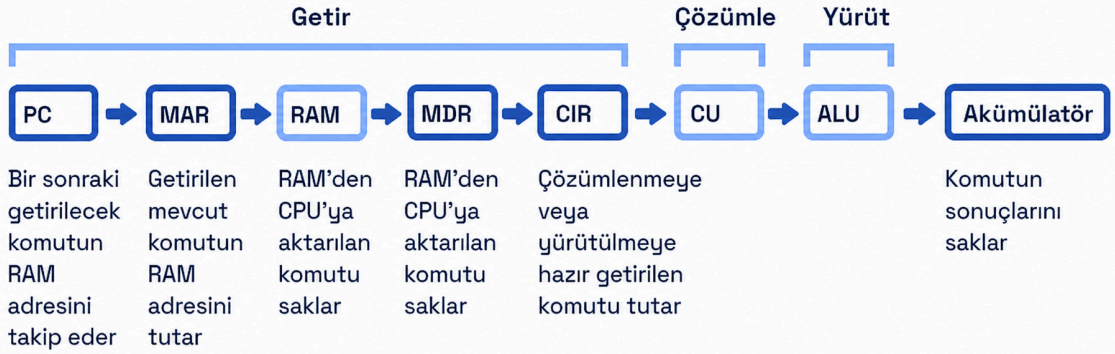
- **LDI (Load Immediate):** Belirlenen bir veriyi doğrudan akümülatöre yükler.
- **ADD / SUB:** Akümülatördeki mevcut veri ile bellekten gelen veriyi işleyerek sonucu tekrar akümülatöre yazar.
- **STORE:** Akümülatördeki değeri veri belleğinin (RAM) belirtilen adresine kaydeder.
- **HALT:** İşlemcinin çalışmasını güvenli bir şekilde durdurur ve **halted** sinyalini tetikleyerek simülasyonu sonlandırır.

Bu sadeleştirilmiş ISA, donanımın doğrulanması sırasında hata ayıklama (debugging) sürecini kolaylaştırırken, ASIC sentezinde zamanlama kısıtlarının (timing closure) daha rahat aşılmasını sağlamaktadır.

3.2.3 Kontrol Birimi ve Sonlu Durum Makinesi (FSM)

İşlemcinin koordinasyon merkezi olan Kontrol Birimi, senkron bir **Sonlu Durum Makinesi (FSM)** olarak tasarlanmıştır. Her bir komut çevrimi, saat sinyaliyle (clock) tetiklenen üç ana fazda tamamlanır:

Getir-Yürüt Döngüsünde Yer Alan Yazmaçlar



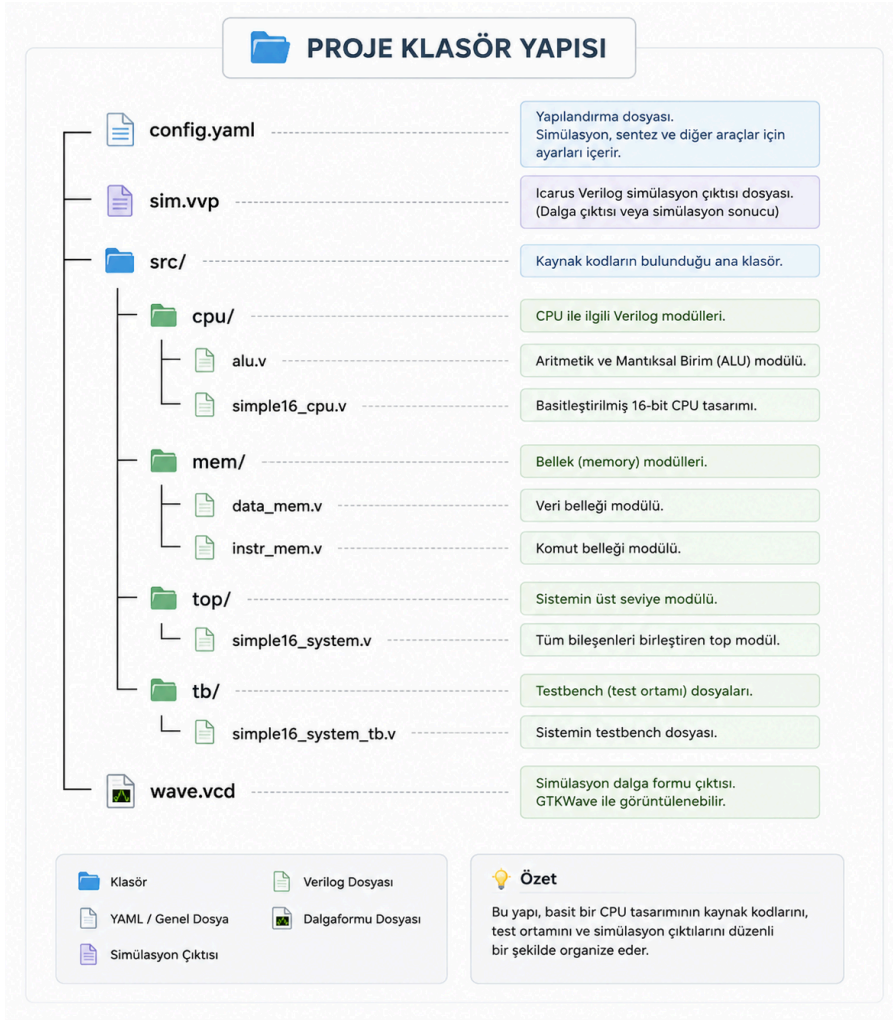
1. **Fetch (Getir):** Program Sayacı (PC) tarafından gösterilen adresteki komut, komut belleğinden (ROM) çekilir.
2. **Decode (Çöz):** Çekilen 16-bitlik verinin en anlamlı bitleri (opcode) ayrıştırılarak hangi işlemin yapılacağı ve hangi kontrol sinyallerinin (ALU seçimi, bellek yazma izni vb.) aktif edileceği belirlenir.
3. **Execute (Yürüt):** Belirlenen operasyon ALU üzerinde gerçekleştirilir, sonuçlar yazmaçlara veya belleğe aktarılır ve PC bir sonraki adres için güncellenir.

3.3 Donanım Gerçekleştirilmesi ve Modüler Yapı

Teorik olarak kurgulanan 16-bit mikroişlemci mimarisi, donanım tanımlama dili Verilog kullanılarak modüler bir yapıda gerçekleştirilmiştir. Bu modülerlik, hata ayıklama süreçlerini kolaylaştırmanın yanı sıra, ASIC tasarım akışında (OpenLane) her bir donanım bloğunun fiziksel olarak doğru konumlandırılmasını sağlamaktadır.

3.3.1 Dosya Hiyerarşisi ve Organizasyon

Sistemin kaynak kodları, fonksiyonel görevlerine göre kategorize edilmiş bir dizin yapısında tutulmaktadır.



Şekil 3.X'te görüldüğü üzere proje; CPU çekirdeği, bellek birimleri ve test katmanları olarak ayrıştırılmıştır.

- **src/cpu/**: İşlecimin hesaplama ve kontrol mantığını içerir. Mantıksal operasyonlar (alu.v) ve komut yürütme döngüsü (simple16_cpu.v) bu dizindedir.
- **src/mem/**: Harvard mimarisinin gereği olan ayrık bellek bloklarını (instr_mem.v, data_mem.v) barındırır.
- **src/top/**: Tüm alt modüllerin birleştirildiği simple16_system.v dosyasını içerir.
- **tb/**: Doğrulama süreçlerinde üretilen simülasyon çıktıları (wave.vcd, sim.vvp) ve test senaryolarını barındırır.

3.3.2 Verilog Modüllerinin Fonksiyonel Tanımı ve Kod Yapısı

Sistemi oluşturan her bir modül, sentezlenebilir (synthesizable) RTL standartlarına uygun olarak tasarlanmıştır:

A. Aritmetik Mantık Birimi (alu.v): İşlemcinin veri işleme çekirdeğidir. Kontrol ünitesinden gelen 3-bitlik op sinyallerine göre toplama, çıkarma ve mantıksal işlemleri gerçekleştirir. Gecikme sürelerini minimize etmek amacıyla tamamen kombinyasyonel bir mantıkla tasarlanmış, case yapısı kullanılarak sentez motorunun (Yosys) en düşük alanla en yüksek hızı elde etmesi hedeflenmiştir.

B. Kontrol ve Veri Yolu Ünitesi (simple16_cpu.v): İşlemcinin "beyni" olarak işlev görür. Bölüm 3.2.3'te tanımlanan FSM (Sonlu Durum Makinesi) burada kodlanmıştır. Akümülatör (ACC) ve Program Sayacı (PC) gibi iç yazmaçlar bu modül içerisinde reg tipinde tanımlanmış; bellekler arası veri trafiğini koordine eden kontrol sinyalleri, saat sinyalinin yükselen kenarında (posedge clk) tetiklenecek şekilde kurgulanmıştır.

C. Komut Belleği (instr_mem.v): Sadece komutların saklandığı salt okunur (read-only) bellek birimidir. İçerisinde, tasarımın işlevselliğini test etmek amacıyla önceden tanımlanmış 16-bitlik bir makine kodu seti barındırır. Sentez aşamasında bu birim, SKY130 kütüphanesindeki standart hücrelere haritalanarak fiziksel yerleşimi yapılır.

D. Veri Belleği (data_mem.v): Çalışma anındaki değişkenlerin depolandığı RAM birimidir. CPU tarafından üretilen we (write enable) sinyaline göre veri yazma veya okuma işlemlerini gerçekleştirir. ASIC akışında bu birimin alan kaplama oranı (utilization), işlemcinin toplam çip boyutunu belirleyen temel unsurlardan biridir.

E. En Üst Seviye Sistem Entegrasyonu (simple16_system.v): İşlemci ve belleklerin bir araya getirilerek birbirine bağlandığı en üst (top-level) modüldür. Bu modül, fiziksel tasarım akışının ana giriş noktasıdır. Dış dünyaya sadece clk, reset, halted ve akümülatör durumunu izlemek için kullanılan acc_out sinyallerini açarak sistemi bir bütün halinde temsil eder.

3.4 Web Tabanlı Platform ve Agentic AI Entegrasyonu

Geleneksel ASIC tasarım süreçlerinde karşılaşılan karmaşık log çıktıları ve parametre optimizasyonu sorunlarını çözmek amacıyla, EDA araçlarını otonom bir şekilde yöneten çok katmanlı bir yazılım platformu geliştirilmiştir. Bu platform, kullanıcı ile düşük seviyeli EDA araçları arasında akıllı bir köprü görevi görmektedir.

3.4.1 Next.js Ön Yüz ve Kullanıcı Deneyimi (UX) Mimarisi

Sistemin kullanıcı ile etkileşime girdiği katman, modern bir web framework'ü olan **Next.js** kullanılarak tasarlanmıştır. Bu katmanın temel görevleri şunlardır:

- **Dinamik Parametre Girişi:** Kullanıcı, fiziksel tasarım için gerekli olan clock_period, utilization ve die_area gibi kritik parametreleri kullanıcı dostu formlar üzerinden sisteme iletebilmektedir.
- **Gerçek Zamanlı Log Takibi:** Python backend üzerinden gelen OpenLane işlem günlükleri (logs), WebSocket veya anlık API sorguları ile kullanıcı arayüzüne yansıtılmakta; böylece tasarımın hangi aşamada (Sentez, Floorplan, Routing vb.) olduğu görsel olarak izlenebilmektedir.
- **AI Öneri Paneli:** Yapay zekâ ajanı tarafından üretilen optimizasyon önerileri ve hata çözümleri, Next.js arayüzünde kullanıcıya onay mekanizmasıyla sunulmaktadır.

3.4.1 Python Backend ve Ajan Tabanlı (Agentic AI) Karar Destek Sistemi

Platformun "akıl" katmanı, Python dili ile geliştirilmiş ve gelişmiş dil modelleri (LLM) ile entegre edilmiş bir otonom ajan sistemidir. Bu katman şu alt bileşenlerden oluşmaktadır:

- **Otonom Karar Mekanizması (Agent):** Klasik betiklerin (scripts) aksine, geliştirilen yapay zekâ ajanı, EDA araçlarından gelen çıktıları anlamlandırabilme yeteneğine sahiptir. Örneğin, bir "Routing Congestion" (yönlendirme tıkanıklığı) hatası aldığında, ajan bu hatanın kaynağını analiz ederek FP_CORE_UTIL parametresini düşürmeyi veya DIE_AREA değerini büyütmeyi otonom olarak teklif edebilmektedir.
- **Bilgi Tabanı ve RAG (Retrieval-Augmented Generation):** OpenLane dokümantasyonu, SKY130 PDK kılavuzları ve geçmiş hata-çözüm kayıtları bir vektör veri tabanına indekslenmiştir. RAG mimarisi sayesinde ajan, karşılaştığı teknik hataları bu bilgi tabanında sorgulayarak, halüsinasyon riskini minimize eden ve teknik doğruluğu yüksek çözümler üretmektedir.
- **EDA Araç Entegrasyonu:** Python backend, Docker konteynerleri içerisinde çalışan OpenLane akışını alt süreçler (subprocesses) aracılığıyla yönetir. Ajanın aldığı kararlar doğrudan config.yaml dosyasına yansıtılmakta ve tasarım akışı otomatik olarak yeniden başlatılabilmektedir.

3.5 ASIC Fiziksel Tasarım Süreci (OpenLane Flow)

Donanım tasarımı ve yazılım entegrasyonu tamamlanan 16-bit işlemci mimarisinin silikon seviyesinde gerçekleştirilmesi amacıyla, açık kaynaklı OpenLane fiziksel tasarım akışı kullanılmıştır. Bu süreç, RTL seviyesindeki mantıksal tanımın, üretim kurallarına (Design Rules) uygun bir maske dosyasına (GDSII) dönüştürülmesini kapsamaktadır.

3.5.1 SkyWater 130nm PDK ve Teknolojik Parametreler

Fiziksel tasarımın hedefi olarak, Google ve SkyWater iş birliğiyle sunulan **SkyWater 130 nm (SKY130)** süreç tasarım kiti (PDK) tercih edilmiştir. 130 nm teknolojisi, akademik çalışmalar için yeterli karmaşıklık düzeyini desteklerken, açık PDK desteği sayesinde DRC (Design Rule Check) ve LVS (Layout Versus Schematic) süreçlerinin otonom olarak doğrulanmasına imkân tanımaktadır. Tasarımda, alan verimliliğini maksimize etmek amacıyla yüksek yoğunluklu (High Density - HD) standart hücre kütüphanesi kullanılmıştır.

3.5.2 Caravel Harness ve IO Pad Wrapper Entegrasyonu (Tape-out Hazırlığı)

Bir tasarımın sadece GDSII dosyasının üretilmesi, fiziksel üretim (tape-out) için yeterli değildir. Silikonun dış dünya ile haberleşebilmesi ve test edilebilmesi için bir "çatı" sisteme ihtiyaç vardır. Bu çalışmada, Efabless tarafından sunulan **Caravel SoC Harness** yapısı kullanılmıştır.

- **Caravel SoC Harness:** Tasarımın içine yerleştirildiği, içerisinde yönetim işlemcisi, güç yönetimi ve giriş/çıkış (I/O) birimlerini barındıran bir "şasi" görevi görür.
- **IO Pad Wrapper & Verilog Wrapper:** Tasarlanan 16-bit işlemci, Caravel'in kullanıcı proje alanına (user_project_wrapper) entegre edilmiştir. **IO Path Wrapper** katmanı olmadan, işlemcinin pinlerinin fiziksel pad'lere eşlenmesi ve voltaj seviyelerinin ayarlanması mümkün değildir. Bu yapı, işlemcinin sinyallerini dış dünyadaki fiziksel bacaklara güvenli bir şekilde bağlayan kritik köprüdür. Bu entegrasyon yapılmadığı takdirde tasarımın "üretilebilir" (tape-out ready) olması mümkün değildir.

3.5.3 Adım Adım Konfigürasyon Stratejisi (Config.yaml)

Tasarımın fiziksel dünyaya aktarılması sürecinde kullanılan **config.json/yaml** dosyası, otonom ajanın müdahale edebileceği parametrik bir hiyerarşiyle yapılandırılmıştır. Her aşama, donanımın PPA (Power-Performance-Area) metriklerini doğrudan etkileyen spesifik flag'ler içerir:

Aşama 0-1: Proje Tanımlama ve Zamanlama Kısıtları Bu aşamada tasarımın kimlik bilgileri (**DESIGN_NAME: simple16_system**) ve temel saat kısıtları tanımlanmıştır. İşlemcinin kararlı çalışması için **CLOCK_PERIOD** değeri 10.0 ns (100 MHz) olarak set edilmiş, **CLOCK_PORT** girişi fiziksel port ile eşlenmiştir.

```
# =====
# 16-BIT SIMPLE CPU – OpenLane ASIC Konfigürasyonu
# Sky130 PDK için optimize edilmiş ayarlar
# =====

#
# AŞAMA 0: TEMEL TASARIM BİLGİLERİ
#
DESIGN_NAME: simple16_system

VERILOG_FILES:
- src/cpu/alu.v
- src/cpu/simple16_cpu.v
- src/mem/instr_mem.v
- src/mem/data_mem.v
- src/top/simple16_system.v

#
# AŞAMA 1: CLOCK (SAAT SİNYALİ) KISITLAMALARI
# - CLOCK_PERIOD: Düşük = daha hızlı ama daha zor timing closure
# - 10ns = 100MHz, 5ns = 200MHz
#
CLOCK_PORT: clk
CLOCK_PERIOD: 10.0 # ns – 100MHz hedef frekans

#
# SKY130A PDK SPESİFİK AYARLAR
#
pdk::sky130A:
```

Şekil 3.X: Proje ismi ve temel zamanlama parametrelerinin tanımlanması (0-1. aşama).

Aşama 2-3: Sentez Stratejisi ve Optimizasyon Mantıksal kapıların haritalanması sürecinde **SYNTH_STRATEGY** parametresi ile alan ve gecikme arasında denge kurulmuştur. **SYNTH_MAX_FANOUT** değeri 10 olarak belirlenerek, sinyal iletim yollarındaki yük dengelenmiş ve elektromigrasyon riskleri minimize edilmiştir.

```
#
# AŞAMA 2: STANDART HÜCRE KÜTÜPHANESİ SEÇİMİ
#
# Seçenekler:
#   - sky130_fd_sc_hd   : High Density (varsayılan, küçük alan)
#   - sky130_fd_sc_hs   : High Speed (daha hızlı, daha büyük)
#   - sky130_fd_sc_ms   : Medium Speed (dengeli)
#   - sky130_fd_sc_ls   : Low Speed (düşük güç tüketimi)
#   - sky130_fd_sc_hdll : High Density Low Leakage
#
STD_CELL_LIBRARY: sky130_fd_sc_hd
```

```
#
# AŞAMA 3: SENTEZ (SYNTHESIS) KISITLAMALARI
#
# MAX_FANOUT_CONSTRAINT: Bir çıkışın besleyebileceği maksimum giriş sayısı
#   - Düşük değer = daha iyi timing, daha fazla buffer
#   - Yüksek değer = daha az alan, potansiyel timing sorunları
#
# CLOCK_UNCERTAINTY_CONSTRAINT: Saat belirsizliği (jitter + skew)
#   - Genelde clock period'un %2-5'i
#
# SYNTH_STRATEGY: Sentez optimizasyon stratejisi
#   - AREA 0-3: Alan optimizasyonu (0 = en agresif)
#   - DELAY 0-4: Hız optimizasyonu (0 = en agresif)
#
MAX_FANOUT_CONSTRAINT: 6
CLOCK_UNCERTAINTY_CONSTRAINT: 0.2 # ns
SYNTH_STRATEGY: "AREA 0" # Maksimum alan küçültme
```

Şekil 3.Y: Sentez motoru (Yosys) kısıtları ve fanout limitleri (2-3. aşama).

Aşama 4: Taban Planlama (Floorplanning) ve Alan Yönetimi Fiziksel tasarımın sınırlarını belirleyen bu aşamada, yonga boyutu (**DIE_AREA**) 150x150 μm^2 olarak sabitlenmiş ve çekirdek alanı (**CORE_AREA**) buna göre oranlanmıştır. **FP_IO_MODE** parametresiyle giriş-çıkış pinlerinin çip kenarlarına dağılımı (random/balanced) kontrol edilmiştir.

```

FP_SIZING: absolute
DIE_AREA: [0, 0, 150, 150]      # 150x150 µm toplam alan (liste formatı)
CORE_AREA: [10, 10, 140, 140]   # 130x130 µm aktif alan
FP_CORE_UTIL: 50                 # %50 kullanım (küçük tasarım için uygun)
FP_ASPECT_RATIO: 1               # Kare çip

```

```

# =====
# AŞAMA 4: FLOORPLANNING (ÇİP YERLEŞİM PLANI)
# =====
# FP_SIZING: "relative" veya "absolute"
#   - relative: Otomatik boyut hesaplama
#   - absolute: Manuel DIE_AREA ve CORE_AREA belirleme
#
# DIE_AREA: Toplam çip alanı [x0, y0, x1, y1] (µm) - LİSTE FORMATI
# CORE_AREA: Logic yerleşim alanı [x0, y0, x1, y1] (µm)
#   - Die kenarlarından 10-20µm boşluk bırakılmalı (I/O için)
#
# FP_CORE_UTIL: Çekirdek kullanım oranı (%)
#   - %30-40: Rahat routing, iyi timing
#   - %50-60: Dengeli
#   - %70+: Agresif, routing zor olabilir
#
# FP_ASPECT_RATIO: En/boy oranı
#   - 1 = kare çip
#   - 2 = dikdörtgen (genişlik = 2 * yükseklik)
# =====

```

Şekil 3.Z: Taban planlama kısıtları ve I/O pin yerleşim stratejisi (4. aşama).

Aşama 5-6: Standart Hücre Yerleştirme ve CTS Hücrelerin çekirdek alanına yerleşimi sırasında **PL_TARGET_DENSITY** değeri %50 (0.5) tutularak, yönlendirme (routing) aşamasında oluşabilecek tıkanıklıkların (congestion) önüne geçilmiştir. Saat Ağacı Sentezi (CTS) aşamasında ise **CLOCK_TREE_SYNTH: 1** aktif edilerek tüm saklayıcılara saat sinyalinin eş zamanlı ulaşması sağlanmıştır.

```
# =====
# AŞAMA 5: PLACEMENT (YERLEŞTİRME) KISITLAMALARI
# =====
# PL_TARGET_DENSITY_PCT: Hedef yerleştirme yoğunluğu
#   - FP_CORE_UTIL'den biraz yüksek olmalı
#   - Çok yüksek = routing congestion
#
# PL_ROUTABILITY_DRIVEN: Routing-aware yerleştirme
#   - true = aktif (önerilir)
#
# PL_TIME_DRIVEN: Timing-aware yerleştirme
#   - true = kritik yolları optimize et
# =====
PL_TARGET_DENSITY_PCT: 55
PL_ROUTABILITY_DRIVEN: true
PL_TIME_DRIVEN: true
```

```
# =====
# AŞAMA 6: CLOCK TREE SYNTHESIS (CTS)
# =====
# RUN_CTS: Clock Tree Synthesis çalıştır
#
# CTS_CLK_MAX_WIRE_LENGTH: Clock ağacında maksimum tel uzunluğu (µm)
#   - Kısa = daha iyi skew, daha fazla buffer
# =====
RUN_CTS: true
CTS_CLK_MAX_WIRE_LENGTH: 200
```

Şekil 3.W: Hücre yerleşim yoğunluğu ve CTS aktivasyonu (5-6. aşama).

Aşama 7: Fiziksel Yönlendirme ve Nihai GDSII Üretimi Akışın son adımında, **GRT_ALLOW_CONGESTION** parametresi "false" tutularak üretim kurallarına aykırı bir yönlendirme yapılması engellenmiştir. **DETAILED_ROUTING** motoru (TritonRoute) ile metal katmanları arası bağlantılar tamamlanmış ve nihai GDSII maske dosyası üretilmiştir.

Şekil 3.Q: Yönlendirme stratejisi ve tasarım kuralı (DRC) hassasiyeti (7. aşama).

```
# =====
# AŞAMA 7: ROUTING (BAĞLANTI) KISITLAMALARI
# =====
# DRT_THREADS: Paralel routing için thread sayısı
#   - CPU çekirdeğinize göre ayarlayın
#
# DRT_OPT_ITERS: Detaylı routing optimizasyon iterasyonu
#   - Yüksek = daha iyi sonuç, daha uzun süre
#
# GRT_ALLOW_CONGESTION: Global routing tıkanıklığa izin ver
#   - false = daha güvenli (önerilir)
# =====
DRT_THREADS: 4
DRT_OPT_ITERS: 64
GRT_ALLOW_CONGESTION: false
```

3.5.4. Fiziksel Tasarımın Tamamlanması ve GDSII Çıktısı

Konfigürasyon stratejilerinin (Aşama 0-7) OpenLane pipeline akışı üzerinde başarıyla koşturulması neticesinde, tasarlanan 16-bit işlemcinin fiziksel yerleşim dosyası (GDSII) üretilmiştir. Bu aşama, dijital tasarımın yarı iletken üretimine hazır hale geldiği "Sign-off" (onay) noktasını temsil etmektedir.

Açık kaynaklı EDA araç zincirinin (Magic ve KLayout) sunduğu fiziksel doğrulama yetenekleri sayesinde, tasarımın SkyWater 130nm PDK üretim kurallarıyla (DRC) tam uyumlu olduğu ve mantıksal netlist ile fiziksel layout arasında bir çelişki bulunmadığı (LVS) belgelenmiştir. Şekil 3.R'de sunulan nihai çıktı, standart hücrelerin çekirdek alanı içerisindeki homojen dağılımını ve metal katmanları arasındaki kompleks yönlendirme ağını (routing) doğrulamaktadır.

```
# =====
# TAHMİNİ SONUÇLAR (16 kelime RAM ile):
# =====
# - Flip-flop sayısı : ~340 (16x16 RAM + CPU regs)
# - Toplam alan : ~150x150 µm = 22,500 µm²
# - Hedef frekans : 100 MHz
# - Teknoloji : Sky130 (130nm)
# =====
```

Şekil 3.R: `simple16_system` mimarisinin OpenLane akışı sonucunda beklediğimiz sonuçlar

3.6. Yapay Zekâ Modelinin Özelleştirilmesi ve RAG Mimarisi Eğitimi

Proje kapsamında geliştirilen otonom ASIC tasarım sisteminin kalbi, doğal dil işleme yeteneklerini spesifik donanım mühendisliği uzmanlığı ile birleştiren yapay zekâ modelidir. Kullanılan yerel büyük dil modelleri (Ollama tabanlı), genel amaçlı metin üretimi için tasarlanmış olup doğrudan ağırlıklarının eğitilmesi (fine-tuning) yerine, projenin ihtiyaç duyduğu uzmanlık bilgisiyile donatılması için Retrieval-Augmented Generation (RAG) mimarisiyle desteklenmiştir.

3.6.1. RAG Mimarisinin Seçimi ve Bağlam Yönetimi

Yapay zekâ modeline Verilog kurallarını öğretmenin bir yolu olan **Modelfile** içerisine kod örnekleri gömmek, modelin 32k ile 128k arasında değişen bağlam penceresini (Context Window) doldurarak yeni kullanıcı soruları için yeterli hafıza bırakmama riskini taşımaktadır. Ayrıca statik olarak gömülen çok büyük veri setleri, ilk token üretim (Time to First Token) süresini ciddi şekilde uzatmaktadır.

Bu donanımsal ve mantıksal sınırları aşmak amacıyla; yüzlerce Verilog dosyasının ve EDA dokümantasyonunun dinamik olarak tarandığı, kullanıcının komutuna en uygun kod parçalarının anlık olarak modele iletildiği RAG tabanlı bir ajan (Agent) mimarisi kurgulanmıştır. Bu mimaride, otonom sistem ile vektör veritabanını entegre eden yerleşik bir yönetim arayüzü tercih edilmiştir.

3.6.2. Eğitim Veri Setinin (Knowledge Base) Oluşturulması ve Stratejisi

Ajanın ASIC tasarımı konusunda uzmanlaşmasını sağlamak üzere **/srv/rag-learning** dizini altında kapsamlı bir veri seti toplanmış ve belirli kategoriler altında sisteme yüklenmiştir. Eğitim verileri şu ana kaynaklardan derlenmiştir:

- **Haberleşme ve Veri Yolları (01_protocols):** AXI4, AXI-Stream, Gigabit Ethernet MAC, PCI Express, UART ve I2C gibi endüstri standardı arayüz protokollerine ait saf Verilog/SystemVerilog kütüphaneleri.
- **Standart Tasarım Yapıları (02_standard_libs):** Sistemin tekerleği yeniden icat etmesini önlemek amacıyla projeye dahil edilen parametrik FIFO'lar, saat bölgeleri arası geçiş (CDC) senkronizörleri ve bellek yapıları barındıran **basejump_stl** ve **surf** gibi silisyumda doğrulanmış yapıtaşları.
- **İşlemci Mimarileri ve Doğrulama (03_verification):** Dünyanın en küçük seri RISC-V işlemcisi SERV, SoC tasarım şablonu VeeRwolf ve LLM'ler için özel hazırlanan Verilog-eval hata ayıklama problemleri.

- **EDA Otomasyon Belgeleri (04_eda_automation):** Sistemin terminalden dönen logları okuyup kendini onarabilmesi (self-healing) için eklenen OpenLane konfigürasyonları, Yosys kuralları ve Verilator hata kılavuzları.

Bu devasa veri setinin RAG tarafından doğru anlamlandırılabilmesi için "Parçalama (Chunking)" stratejisi uygulanmıştır. Sisteme yüklenen ham kod blokları, doğrudan bulundukları README dosyaları ve teknik kılavuzlarıyla eşleştirilmiştir. Ayrıca sistemin simülasyon kodları ile sentez kodlarını birbirine karıştırmasını engellemek amacıyla yalnızca sentezlenebilir (synthesizable) Verilog kodları veri setine dahil edilerek kalite artırılmıştır.

3.6.3. Verilerin Vektörleştirilmesi (Embedding) ve Qdrant İndeksleme Süreci

Hazırlanan donanım veri setleri, matematiksel temsiller olan vektörlere dönüştürülerek (embedding) vektör veritabanına indekslenmiştir. Bu işlem sırasında vektör boyutu (dimensionality) 768 olarak belirlenmiştir.

- Eski nesil 384 boyutlu vektörler, AXI zamanlamaları ve setup/hold ilişkileri gibi yoğun teknik Verilog terimlerini birbirine karıştırma riski taşıdığı için tercih edilmemiştir.
- 3072 boyutlu vektörlerin ise matris çarpımlarını ağırlaştırarak arama sürelerini uzatması ve bellek israfına (RAM tüketimine) yol açması göz önünde bulundurulmuştur.
- Bu bağlamda 768 boyut, kodun yapısal mantığını ve hiyerarşisini kusursuzca ayırt edebilen "altın oran" olarak sisteme entegre edilmiştir.

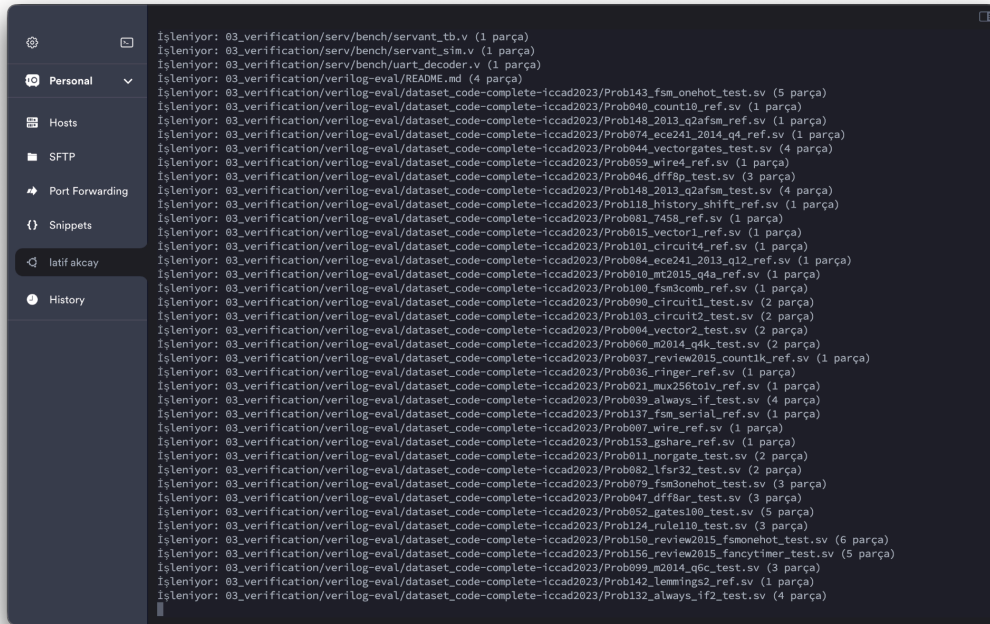
Geliştirilen RAG modelinin eğitim (vektörleştirme) işlemi yaklaşık iki saat sürmüş ve ağırlıklı graf tabanlı Qdrant vektör veritabanı üzerinde 21.000 düğüm (node) başarıyla oluşturulmuştur. Sisteme entegre edilen Qdrant altyapısı, sunucunun 6333 portu üzerinden sunulan web arayüzü ile anlık olarak izlenmiştir. Bu kontrol paneli sayesinde graf vektörleri üzerinde görsel sorgulama yapılmış, atılan spesifik (SQL tarzı) aramaların anlık vektör karşılıkları elde edilmiştir.

Eğitim sürecinde, sisteme sunulan verilerin çok büyük bir kısmı başarıyla indekslenmiş olsa da; boyutu aşırı büyük olan bazı kaynak dosyalar, sistemin belirlediği maksimum context bağlamını aştığı için güvenli bir şekilde reddedilmiş ve indeksleme dışı bırakılmıştır.

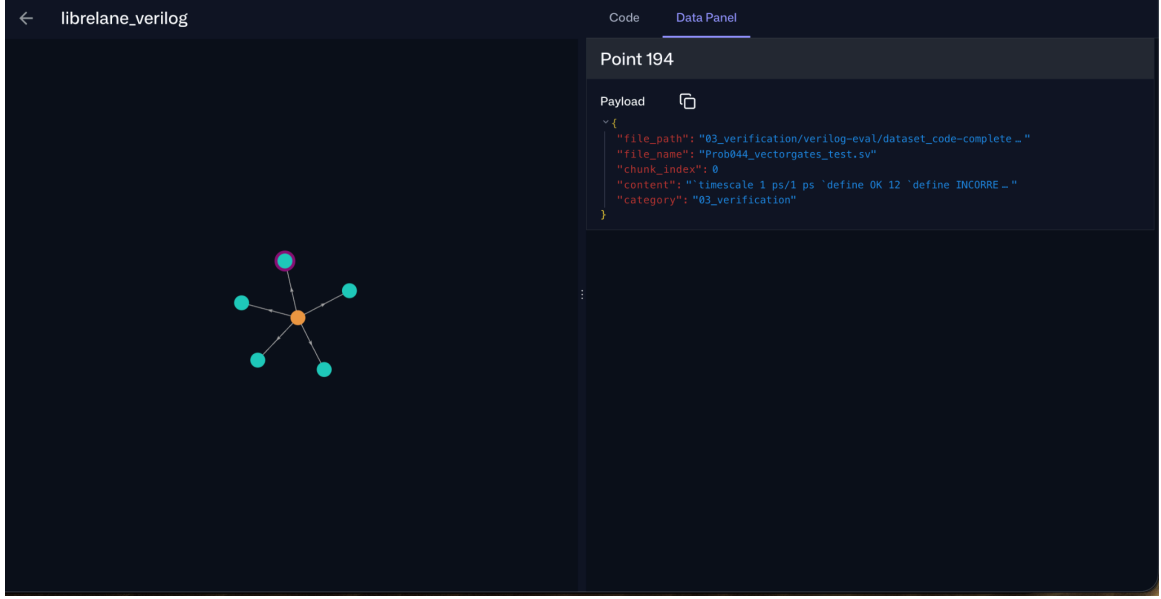
Collection Info Trigger Optimizers

```
{
  "status": "green" // Current state of the collection.
  "optimizer_status": "ok" // Current state of the collection
  "indexed_vectors_count": 0 // Approximate number of indexed vectors in the collection.
  "points_count": 131 // Approximate number of points (vectors + payloads) in collect...
  "segments_count": 7 // Number of segments in collection.
  "config": {
    "params": {
      "vectors": {
        "size": 768 // Size of a vectors used
        "distance": "Cosine" // Type of internal tags, build from payload Distance function -
      }
      "shard_number": 1 // Number of shards the collection has
      "replication_factor": 1 // Number of replicas for each shard
      "write_consistency_factor": 1 // Defines how many replicas should apply the operation for us -
      "on_disk_payload": true // If true - point's payload will not be stored in memory.
    }
    "hnsw_config": {
      "m": 16 // Number of edges per node in the index graph.
      "ef_construct": 100 // Number of neighbours to consider during the index building.
      "full_scan_threshold": 10000 // Minimal size threshold (in KiloBytes) below which full-scan -
      "max_indexing_threads": 0 // Number of parallel threads used for background index buildin...
      "on_disk": false // Store HNSW index on disk.
    }
    "optimizer_config": {
      "deleted_threshold": 0.2 // The minimal fraction of deleted vectors in a segment, requir-
      "vacuum_min_vector_number": 1000 // The minimal number of vectors in a segment, required to perf-
      "default_segment_number": 0 // Target amount of segments optimizer will try to keep.
      "max_segment_size": NULL
      "memmap_threshold": NULL
      "indexing_threshold": 10000 // Maximum size (in kilobytes) of vectors allowed for plain ind-
      "flush_interval_sec": 5 // Minimum interval between forced flushes.
      "max_optimization_threads": NULL
      "prevent_unoptimized": NULL
    }
    "wal_config": {
      "wal_capacity_mb": 32 // Size of a single WAL segment in MB
      "wal_segments_ahead": 0 // Number of WAL segments to create ahead of actually used ones
      "wal_retain_closed": 1 // Number of closed WAL segments to keep
    }
    "quantization_config": NULL
  }
  "payload_schema": {}
  "update_queue": {
    "length": 0 // Number of elements in the queue
  }
}
```

Şekil 3.2: Yeni başlayan RAG eğitimi ve eğitim parametrelerimiz



Şekil 3.3: Devam eden RAG eğitimi



Şekil 3.4: Qdrant vektör veritabanı arayüzünde indekslenen Verilog kod bloklarının vektör uzayındaki anlamsal yakınlık (Semantic Similarity) ilişkilerini gösteren graf temsili.

4. MATERİYAL VE METOT

Bu bölümde, çalışmanın gerçekleştirilmesi sürecinde kullanılan donanım, ağ ve yazılım materyalleri ile projenin hayata geçirilmesinde izlenen metodolojik yaklaşım ayrıntılı olarak açıklanmaktadır. Tasarlanan 16-bit CPU mimarisinin geliştirilmesi, simülasyonu ve doğrulanması aşamalarında kullanılan açık kaynaklı araç zinciri, bu zinciri otomatize eden yerel büyük dil modeli (Local LLM) destekli akıllı asistan altyapısı ve sistemin işleyiş metodu bu kapsamda sunulmaktadır.

4.1 Kullanılan Materyaller

Bu bitirme projesi kapsamında, 16-bit bir işlemci (CPU) mimarisinin tasarımı, doğrulanması ve fiziksel tasarım akışının incelenmesi için çeşitli işletim sistemleri, açık kaynaklı EDA (Elektronik Tasarım Otomasyonu) araçları ve projeye özgü geliştirilen yapay zekâ entegrasyon sistemleri kullanılmıştır.

4.1.1 Donanım ve Ağ Altyapısı

Tasarım, simülasyon ve otonom optimizasyon süreçlerinin farklı platformlarda sorunsuz biçimde yürütülebilmesi, taşınabilirliğin ve tekrarlanabilirliğin doğrulanması amacıyla aşağıdaki donanım bileşenleri ve ağ altyapısı tercih edilmiştir:

- **Kişisel Bilgisayar (İstemci Ortamı):** x86_64 ve ARM64 mimarileri üzerinde, yerel simülasyonların yapılması ve arayüz geliştirmeleri için Unix tabanlı macOS (ana geliştirme ortamı) ve NT çekirdekli Windows (ikincil test ortamı) işletim sistemleri kullanılmıştır.
- **Yapılan Zekâ ve Tasarım İş İstasyonu (Ana Sunucu):** Erzurum Teknik Üniversitesi (ETÜ) tarafından proje kapsamında tahsis edilen; NVIDIA RTX 5070 Ti grafik işlem birimi (GPU) ve 32 GB DDR5 RAM ile donatılmış yüksek performanslı fiziksel iş istasyonudur. Bu donanım, 26 milyar parametrelili yerel dil modelinin (Gemma) akıcı çalıştırılması ve ağır EDA sentez/fiziksel tasarım işlemlerinin koşturulması için ana hesaplama sunucusu (compute server) görevini üstlenmiştir.
- **Tailscale (Güvenli Ağ Katmanı):** Ana sunucunun dış siber tehditlere karşı korunması ve port yönlendirme (port-forwarding) zafiyetlerinin engellenmesi amacıyla kurulan şifreli örgüsel (mesh) VPN altyapısıdır. Geliştiriciler sunucuya bu izole ağ üzerinden SSH protokolü ile erişim sağlamıştır.

4.1.2 Yazılım Materyalleri ve Geliştirme Ortamları

Projenin donanım otomasyonu ve kullanıcı arayüzü katmanlarında tamamen açık kaynak kodlu modern yazılım materyalleri konumlandırılmıştır:

- **Ollama & Model Ekosistemi:** Yerel çıkarım motoru olarak Ollama tercih edilmiştir. Bu altyapı, Hugging Face üzerindeki açık kaynaklı modellerin **GGUF** formatında, sistem kaynaklarına (RTX 5070 Ti) uygun şekilde **kuantize (4-bit/8-bit)** edilerek koşturulmasına olanak tanımıştır. Kuantizasyon işlemi, yüksek parametrelili modellerin (Örn: 80B) bellek kullanımını minimize ederken işlem hızını maksimize etmiştir.
- **Ollama & Gemma (Yapay Zekâ Katmanı):** Sunucu üzerinde tamamen yerel (local) olarak koşturulan Ollama çıkarım motoru ve Google'ın açık kaynaklı 26 milyar parametrelili Gemma dil modeli kullanılmıştır.
- **Python (Ajan ve Otomasyon Katmanı):** EDA araçlarının log çıktılarını analiz eden, süreç kontrolünü sağlayan ve yapay zekâ ajanının karar mekanizmalarını yöneten arka uç (backend) yazılım materyalidir.
- **Next.js & Nginx (Arayüz ve Dağıtım Katmanı):** Tasarımcının süreci grafiksel olarak izlemesini sağlayan ETU LibreLane AI web arayüzü Next.js ile

geliştirilmiş; sunucu üzerindeki ters proxy ve yönlendirme işlemleri Nginx ile optimize edilmiştir.

- **LibreLane & Çekirdek EDA Araçları:** Sentez için **Yosys**, yerleştirme, yönlendirme ve zamanlama analizi için **OpenROAD**, fiziksel doğrulama için ise **Magic** ve **KLayout** araçlarını barındıran uçtan uca tasarım akışıdır. Üretim parametreleri için **SkyWater 130nm PDK** süreç kiti temel alınmıştır.

4.2 Kullanılan Yöntemler (Metot)

Proje kapsamında; geleneksel dijital tasarım metodolojisi, modern yapay zekâ orkestrasyonu ve iteratif parametre optimizasyonu ile birleştirilerek bütüncül bir yöntem izlenmiştir. Bu yaklaşım, donanım mimarisinin tasarımından başlayarak fiziksel üretime hazır GDSII çıktısının alınmasına kadar olan süreci otonom bir döngüye oturtmaktadır.

4.2.1 Modüler RTL Tasarım ve Doğrulama Yaklaşımı

Donanım geliştirme sürecinde "Böl ve Yönet" (Divide and Conquer) felsefesi benimsenmiştir. İşlemci mimarisi; PC, ALU, Yazmaçlar ve Bellek birimleri gibi fonksiyonel açıdan bağımsız alt Verilog modüllerine ayrılmıştır. Her modül, Yazmaç Aktarım Seviyesinde (RTL) bağımsız olarak kodlanmış ve **Icarus Verilog** ile birim testlerine tabi tutulmuştur. Fonksiyonel doğrulama metodu olarak, her komutun (ISA) beklenen çıktıları **GTKWave** üzerinden görselleştirilerek mantıksal tutarlılık denetlenmiştir.

4.2.2 Ajan Tabanlı EDA Akışı ve Log Analiz Metodu

Geleneksel EDA akışlarındaki manuel operasyon yükünü azaltmak amacıyla "Ajan Tabanlı Otomasyon" yöntemi uygulanmıştır. Bu yöntemde Python backend ünitesi, OpenLane akışı sırasında üretilen binlerce satırlık sentez ve yönlendirme loglarını anlık olarak tarar. Metot, kritik hata (Error) veya zamanlama ihlali (Timing Violation) yakalandığında süreci durdurmak yerine; hatayı **Gemma (27B)** modeline bir "context" (bağlam) olarak sunmak ve modelden gelen teknik çözüm önerisini otomatik olarak **config.json** dosyasına işleyerek akışı yeniden başlatmak üzerine kuruludur.

4.2.3 İteratif PPA Optimizasyonu ve Otonom Karar Döngüsü

Çipin Alan (Area), Güç (Power) ve Performans (Performance) metriklerini (PPA) iyileştirmek için iteratif bir metodoloji izlenmiştir. Sistem, fiziksel tasarım kurallarını (DRC) ihlal etmeden en

optimum sonuçlara ulaşana kadar parametreleri (Örn: **FP_CORE_UTIL**, **MAX_FANOUT**) kullanıcı isteğine göre kademeli olarak değiştirir. Bu süreçte yapay zekâ ajanı deneme-yanılma yolu ile hata alana kadar devam etmektedir.

4.2.4 Fiziksel Tasarım ve İşaretleme (Sign-off) Metodolojisi

Doğrulanmış tasarımın fiziksel gerçekleşmesi aşamasında; mantıksal sentez, taban planlama, saat ağacı sentezi (CTS) ve detaylı yönlendirme adımları bir pipeline (boru hattı) mantığıyla işletilmiştir. Nihai aşamada, tasarımın fabrikasyon kurallarına tam uyumunu garanti etmek amacıyla **Magic** ile DRC ve **Netgen** ile LVS (Layout vs Schematic) testleri koşturularak "Sign-off" (onay) prosedürü tamamlanmıştır.

4.2.5 Model Seçim Stratejisi ve Evrimi: Gemma'dan Qwen'e Geçiş

Proje kapsamında kullanılan yapay zekâ ajanının karar verme yeteneği ve hızı, kullanılan model mimarisine doğrudan bağlıdır. Bu süreçte üç temel aşamadan geçilerek donanım-yazılım uyumu optimize edilmiştir:

- **Aşama 1: Google Gemma (27B) ve Performans Darboğazı:** Başlangıç aşamasında tercih edilen Gemma 27B modeli, "**Dense**" (**Ham**) bir mimariye sahip olması nedeniyle her çıkarım işleminde tüm parametrelerini aktif etmektedir. Bu durum, ETÜ iş istasyonu üzerindeki RTX 5070 Ti grafik işlemcisinin VRAM kapasitesini zorlamış ve saniye başına üretilen kelime (token) hızının, otonom bir EDA akışı için yetersiz kalmasına neden olmuştur.
- **Aşama 2: Qwen-2.5 80B ve Halüsinasyon Sorunsalı:** Daha karmaşık mantıksal analizler yapabilmek adına geçilen 80 milyar parametrelili Qwen modeli, ilk etapta yüksek başarı gösterse de, bağlam penceresi (context window) dolduğunda sistem kaynaklarını "**Deadlock**" (**Ölümçül Kilitlenme**) durumuna sokmuştur. Bu aşamada model, Verilog loglarını analiz etmek yerine teknik veriden tamamen koparak "Piri Reis'in keşif haritaları" gibi alakasız konularda İngilizce anlatımlar yapmaya ve Çince karakterler üretmeye (hallucination) başlamıştır. Bu durum, yerel sistemlerde kontrolsüz parametre artışının kararlılığı bozduğunu kanıtlamıştır.
- **Aşama 3: Qwen-2.5 Coder (Kırpılmış/Optimize) ve MoE Çözümü:** Nihai aşamada, kodlama ve donanım tanımlama dilleri (HDL) için optimize edilmiş, **Mixture of Experts (MoE)** tabanlı Qwen-Coder serisine geçilmiştir. Bu model, toplamda yüksek parametre sayısına sahip olsa da, aktif işlem anında sadece 3.5B ila 8B arası parametreyi çalıştıran "uzman katmanları" kullanmaktadır. Bu sayede, Ollama üzerinden sağlanan **4-bit/8-bit**

kuantizasyon desteğiyle birlikte, iş istasyonunda "sıfır dirençle" (minimum gecikme ve yüksek doğrulukla) çalışan stabil bir ajan yapısı kurulmuştur.

4.2.6 Yazılım-Donanım Katmanı Arasındaki Köprü: Ollama API ve Soyutlama

Sistemin esnekliğini ve model bağımsızlığını sağlamak amacıyla, donanım kaynakları ile yapay zekâ ajanı arasına bir **"Model Soyutlama Katmanı" (Model Abstraction Layer)** olarak Ollama API konumlandırılmıştır. Bu metodolojik tercih şu avantajları sağlamıştır:

- **Sıfır Dirençli Model Göçü (Single-Click Migration):** Yapay zekâ ajanının karar mekanizması, Python tabanlı backend kodunda herhangi bir refactoring (kod iyileştirme) ihtiyacı doğmadan tek bir parametre değişikliğiyle güncellenebilmektedir. Bu durum, projenin farklı model mimarileriyle (Gemma, Qwen vb.) test edilmesini "sıfır dirençli" bir hale getirmiştir.
- **Asenkron Çıkarım ve Kaynak Yönetimi:** EDA araçlarının (Yosys, OpenROAD) yoğun CPU/RAM kullanımı ile LLM'in yoğun GPU kullanımı arasındaki denge, Ollama'nın API tabanlı yönetim sistemiyle optimize edilmiştir.
- **Kuantizasyon Odaklı Bellek Yönetimi:** Hugging Face üzerinden temin edilen model ağırlıkları, GGUF formatında 4-bit ve 8-bit kuantizasyon teknikleriyle sisteme dahil edilmiştir. Bu yöntem, 80B gibi devasa modellerin dahi 16GB VRAM kapasitesine sahip yerel iş istasyonunda çalışabilmesine olanak tanıyan temel materyal hazırlık ve uygulama metodudur.

5. BULGULAR VE ANALİZLER

Bu bölümde; tasarlanan 16-bit Simple16_system mimarisinin mantıksal doğruluğu, fiziksel tasarım süreçlerindeki verimliliği ve projeye özgü geliştirilen yapay zekâ tabanlı otonom akışın performans verileri sunulmaktadır. Elde edilen bulgular, sadece bir donanım tasarımının başarısını değil; aynı zamanda yerel dil modellerinin (Local LLM) karmaşık mühendislik süreçlerini optimize etme kapasitesini de analiz etmektedir. Tasarımın RTL seviyesindeki fonksiyonel ispatlarından, fiziksel üretim (GDSII) metriklerine ve yapay zekânın karar mekanizmalarındaki kararlılığına kadar olan tüm veriler, karşılaştırmalı bir perspektifle değerlendirilmiştir.

5.1 Mikroişlemcinin Mimari Doğrulaması & RTL ve Fonksiyonel Doğrulama (TB/GTKWave)

Bu bölümde, tasarlanan 16-bit işlemci sisteminin açık kaynaklı ASIC tasarım akışı kapsamında elde edilen sentez, zamanlama, yerleştirme-yönlendirme ve fiziksel doğrulama sonuçları sunulmakta ve bu sonuçlar ayrıntılı şekilde değerlendirilmektedir. Çalışma kapsamında elde edilen bulgular, tasarımın hem fonksiyonel hem de fiziksel açıdan üretilebilir olduğunu kanıtlamaktadır.

5.1.1 Simple16_system Mimari Doğrulaması ve RTL Simülasyon Analizleri

Fiziksel tasarım akışına (RTL-to-GDSII) başlanmadan önce, tasarlanan işlemci mimarisinin mantıksal ve fonksiyonel olarak hatasız çalıştığının kanıtlanması gerekmektedir. Bu başlık altında; 16-bit CPU mimarisinin donanım özellikleri, Icarus Verilog ile koşturulan testbench simülasyonları ve GTKWave üzerinden gerçekleştirilen dalga formu (waveform) analizleri dökümanite edilmiştir.

5.1.2 16-Bit CPU Fonksiyonel Çalışma Prensipleri

Tasarlanan merkezi işlem birimi, 16-bit genişliğinde bir veri yoluna (data bus) sahip olup tamamen senkron çalışma prensibiyle yapılandırılmıştır. Mimari üzerindeki tüm durum değişimleri ve veri aktarımları küresel bir saat (clock) sinyali ile kontrol edilmekte, sistemin başlangıçtaki kararlı durumu ise asenkron bir reset sinyali vasıtasıyla belirlenmektedir.

İşlemci çekirdeği; komutun bellekten getirilmesi (fetch), çözülmesi (decode) ve ALU üzerinde yürütülmesi (execute) adımlarından oluşan temel donanım döngüsünü donanımsal olarak icra etmektedir. Komut seti mimarisi (ISA) kapsamında; işlemciyi bekleten NOP, bellekten veri çeken LOAD, belleğe veri yazan STORE, saklayıcıya doğrudan değer atayan ADDI ve programı güvenli şekilde sonlandıran HALT gibi temel kontrol ve aritmetik komutlar donanım seviyesinde desteklenmektedir.

5.1.3 RTL Simülasyon Stratejisi ve Testbench Yapısı

İşlemcinin işlevsel doğruluğunu, komut yürütme yeteneğini ve bellek okuma/yazma zamanlamalarını test etmek amacıyla sistem seviyesinde kapsamlı bir testbench ([simple16_system_tb](#)) mimarisi geliştirilmiştir. Geliştirilen doğrulama ortamında; kararlı bir saat sinyali üretilmiş, sınır durumları test etmek için reset senaryoları uygulanmış; CPU, bellek ve ALU alt modüllerinin birbirleriyle olan veri trafiği izlenmiştir.

```

// src/tb/simple16_system_tb.v
`timescale 1ns/1ps

module simple16_system_tb;
    initial begin
        $dumpfile("wave.vcd");
        $dumpvars(0, simple16_system_tb);
    end

    reg clk;
    reg reset;

    wire halted;
    wire [15:0] acc_out;

    // Test edilen sistem
    simple16_system dut (
        .clk      (clk),
        .reset    (reset),
        .halted   (halted),
        .acc_out  (acc_out)
    );

```

Şekil 3.1. simple16_system tasarımının fonksiyonel doğrulaması için oluşturulan testbench (simple16_system_tb) kodu

İşlemcinin fiziksel tasarım kısıtlarında hedef her ne kadar 100 MHz olsa da, fonksiyonel simülasyonların doğruluğu için testbench içerisinde bu saat frekansı periyodik dalga formları üretilerek simüle edilmiştir.

```
// Clock üretimi: 10ns period → 100 MHz
initial begin
    clk = 0;
    forever #5 clk = ~clk;
end
```

Şekil 3.2. Testbench içerisinde 100 MHz saat sinyalinin (clock) üretilmesi

Sistemin kararlı açılış davranışını doğrulamak ve komut setinin belleğe yüklenerek sırayla çalıştırılmasını tetiklemek amacıyla testbench içerisine doğrulama kontrol blokları entegre edilmiştir.

```
// Test senaryosu
initial begin
    $display("Test basliyor...");

    reset = 1;
    #20;
    reset = 0;

    // Maks 200 ns bekleyelim
    wait (halted == 1'b1 || $time > 2000);

    $display("Sim sonu: time=%0t ns, halted=%b, ACC=%0d (0x%04h)",
        $time, halted, acc_out, acc_out);

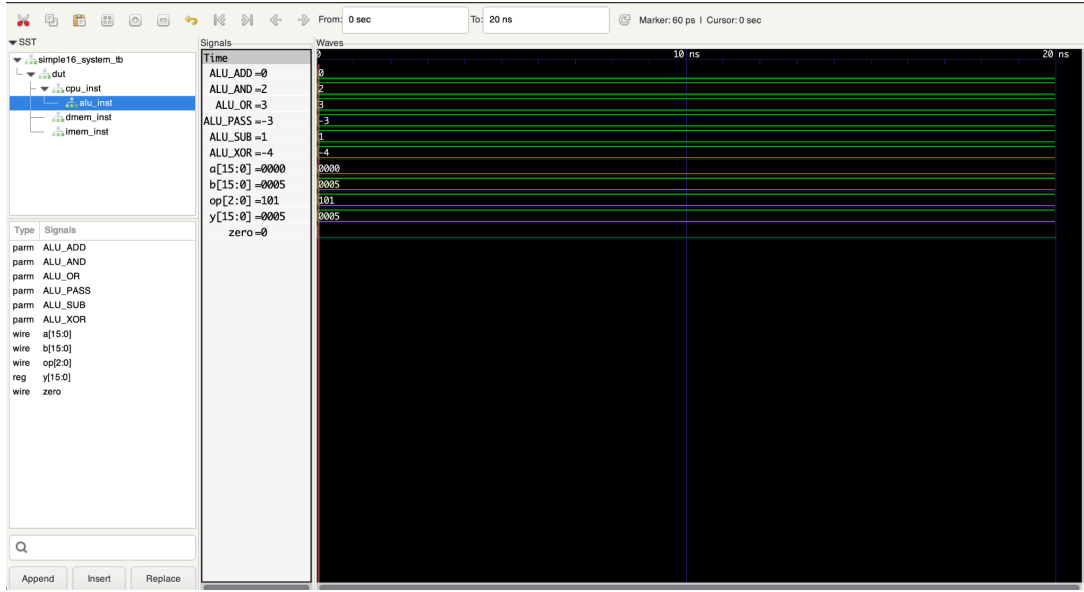
    if (halted && acc_out == 16'd15) begin
        $display("TEST OK: ACC=15 bekledigimiz gibi.");
    end else begin
        $display("TEST FAIL: ACC beklenenden farkli.");
    end

    #20;
    $finish;
end
endmodule
```

Şekil 3.3. Testbench içerisinde reset, test senaryosu ve doğrulama kontrolünün tanımlanması

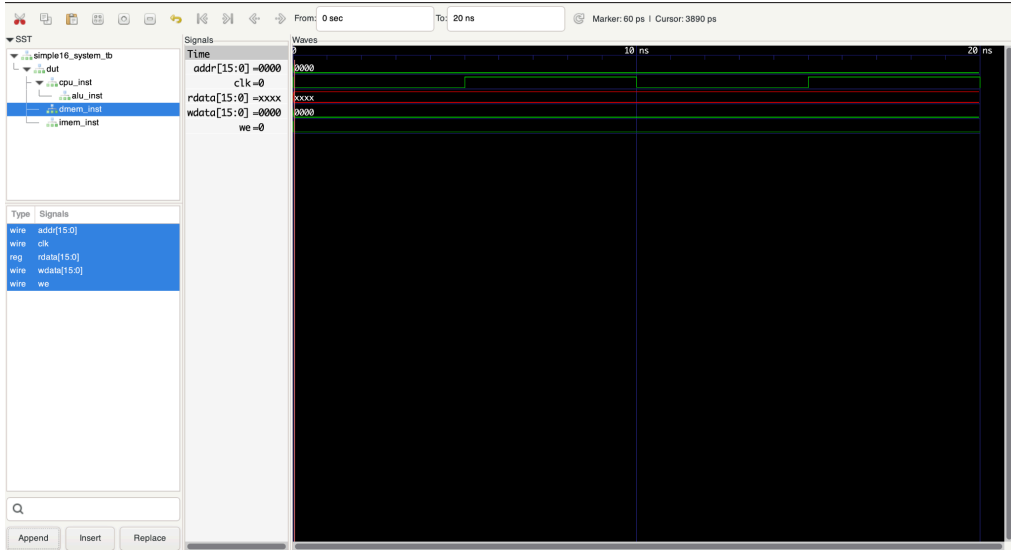
5.1.4 GTKWave ile Dalga Formu (Waveform) Analizleri

Açık kaynaklı **Icarus Verilog** derleyicisi ile koşturulan simülasyonlar sonucunda üretilen değer değişim dökümleri (VCD dosyaları), **GTKWave** aracı üzerinden dalga formu seviyesinde görselleştirilmiştir. İlk olarak, aritmetik ve mantıksal işlemlerin doğruluğunu denetlemek adına ALU modülünün sinyal geçişleri, elde (carry) ve sıfır (zero) bayrakları analiz edilmiştir.,



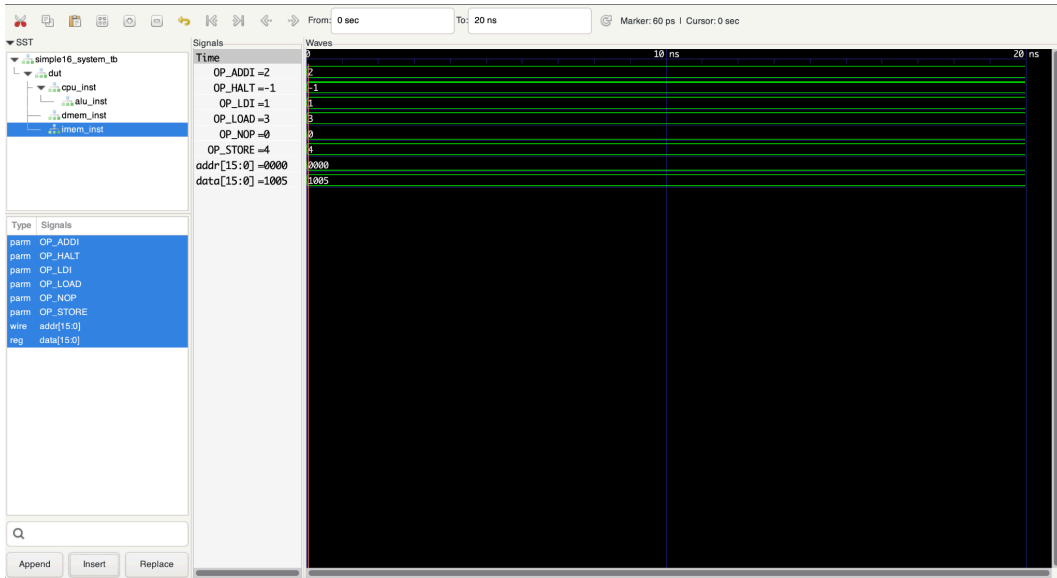
Şekil 3.4. ALU modülüne ait sinyallerin GTKWave üzerinde simülasyon dalga formu ile doğrulanması

İşlecimin veri belleğiyle (Data Memory) olan ilişkisini, yazma yetkisini (write enable) ve adres-veri tutarlılığını doğrulamak adına bellek veri yolu sinyalleri dalga formu üzerinde incelenmiştir.



Şekil 3.5. Veri belleği (data memory) modülüne ait adres, veri ve yazma kontrol sinyallerinin GTKWave üzerinde simülasyon dalga formu ile incelenmesi

Son olarak, komut belleğinden (Instruction Memory) çekilen operasyon kodlarının (opcode) doğruluğu ve program sayacının (PC) ardışık komutları kaçırmadan getirdiği zamanlama grafiği üzerinden teyit edilmiştir.



Şekil 3.6. Komut belleği (instruction memory) modülüne ait opcode ve veri sinyallerinin GTKWave üzerinde simülasyon dalga formu ile doğrulanması

Simülasyon sürecinin başarıyla tamamlandığını, herhangi bir mantıksal çelişki (X veya Z durumları) yaşanmadığını ve HALT komutuyla programın güvenli şekilde sonlandığını gösteren terminal çıktıları konsol üzerinden dökümanite edilmiştir.

```
VCD info: dumpfile wave.vcd opened for output.  
Test basliyor...  
Sim sonu: time=75000 ns, halted=1, ACC=15 (0x000f)  
TEST OK: ACC=15 bekledigimiz gibi.  
tb/simple16_system_tb.v:51: $finish called at 95000 (1ps)
```

Şekil 3.7. Testbench simülasyonu sonucunda sistemin doğru şekilde çalıştığını gösteren konsol çıktısı

5.1.5 Fiziksel Tasarım Akışı (Physical Design Flow)

Fonksiyonel simülasyonları ve mantıksal doğrulaması başarıyla tamamlanan Simple16_system RTL tasarımı, deneysel olarak ASIC fiziksel tasarım akışına dahil edilmiştir. Bu süreç, soyut Verilog kodlarının üretilebilir gerçek yarı iletken geometrilerine (layout) dönüştürülmesini kapsamaktadır. Akış kapsamında şu yöntemsel adımlar izlenmiştir:

- 4.1.1 **Mantıksal Sentez:** Doğrulanmış RTL kodları, **Yosys** sentez motoru kullanılarak hedef kütüphane kapılarına eşlenmiş ve kapı seviyesinde netlist (gate-level netlist) üretilmiştir.
- 5.1.1 **Yerleştirme ve Yönlendirme (P&R):** Elde edilen netlist, **OpenROAD** altyapısı kullanılarak yonga alanı üzerine yerleştirilmiş, saat ağacı sentezi (CTS) koşturulmuş ve hücreler arası metal bağlantılar elektriksel olarak yönlendirilmiştir.
- 6.1.1 **Fiziksel Doğrulama:** Üretim öncesi son aşamada, tasarımın geometrik doğruluğu **Magic** ve **KLayout** araçları vasıtasıyla test edilmiştir. Bu doğrulamada, tasarımın fabrikasyon kurallarına uygunluğu (DRC) ve şematik-layout tutarlılığı (LVS) tam olarak denetlenmiştir.

Tüm bu fiziksel tasarım ve üretim adımlarında, endüstriyel standartlarda bir çıktı elde edebilmek adına hedef üretim teknolojisi olarak **SkyWater 130 nm (sky130A)** süreci ve bu sürece ait tasarım kiti (PDK) referans alınmıştır.

5.1.6 İşlevsel ve Yapısal Süreçlerin Değerlendirilmesi

Bu aşamaya kadar gerçekleştirilen fonksiyonel simülasyonlar, dalga formu analizleri ve temel fiziksel tasarım adımları bir arada değerlendirildiğinde; tasarlanan 16-bit Simple16_system CPU mimarisinin **hem işlevsel (RTL seviyesinde) hem de yapısal (fiziksel tasarım seviyesinde) olarak doğru çalıştığı** kanıtlanmıştır. Icarus Verilog ve GTKWave ile elde edilen çıktılar, işlemcinin komut seti mimarisini (ISA) hatasız yürüttüğünü; Yosys ve OpenROAD süreçleri ise tasarımın mikroçip üretim kurallarına (SKY130) başarıyla uyulanabildiğini ortaya koymuştur.

Kullanılan tamamen açık kaynaklı araç zinciri, bu yönüyle akademik ve endüstriyel donanım geliştirme standartlarıyla tam uyumlu bir zemin sağlamıştır.

5.1.7 Geleneksel Doğrulama Süreçlerinin Sınırları ve Yapay Zekâ İhtiyacı

Geleneksel EDA ve doğrulama akışları başarılı sonuçlar üretse de, bu ham verilerin (binlerce satırlık karmaşık log dosyaları, sentez raporları ve terminal çıktıları) anlamlandırılması tasarımcı üzerinde çok ciddi bir operasyonel yük oluşturmaktadır. Süreç esnasında meydana gelen mikro ölçekli zamanlama ihlallerinin (slack hataları) veya geometrik DRC/LVS hatalarının insan gözüyle manuel olarak taranması zaman alıcı ve hataya açıktır.

Ayrıca, işlemcinin güç, performans ve alan (PPA) metriklerini optimize etmek amacıyla yüzlerce farklı araç parametresinin (flag) deneme-yanılma yoluyla elle ayarlanması, süreci insan iş gücüne aşırı bağımlı hale getirmektedir. Açık kaynaklı araçların sağladığı bu devasa veri yığınının otonom olarak filtrelenmesi, hataların kendi kendini onaran (auto-repair) döngülerle çözülmesi ve parametrelerin en iyi çip çıktısını üretecek şekilde akıllıca optimize edilmesi için geleneksel akışın bütüncül bir yapay zekâ ekosistemiyle desteklenmesi zorunlu bir mühendislik ihtiyacı olarak ortaya çıkmıştır.

5.2 Yapay Zekâ Destekli Otonom Tasarım ve Model Performans Bulguları

Geleneksel EDA akışının yapay zekâ ekosistemine entegre edilmesiyle birlikte, sistemin karar verme, hata ayıklama ve optimizasyon yetenekleri farklı büyük dil modelleri (LLM) üzerinden test edilmiştir. Bu testler, yerel donanım kaynakları ile model parametre sayısının tasarım başarısı üzerindeki etkisini somut verilerle ortaya koymuştur.

5.2.1 Model Karşılaştırmalı Hata Analizi ve Kararlılık Testleri

Proje kapsamında kullanılan modellerin, binlerce satırlık EDA loglarını analiz etme ve çözüm üretme başarısı "Vaka Analizleri" üzerinden değerlendirilmiştir:

- **Google Gemma-27B Analizi:** "Ham" (Dense) mimarisi nedeniyle RTX 5070 Ti iş istasyonunun VRAM kapasitesini sınırlarına kadar zorlamıştır. Sentez hatalarını yakalamada başarılı olsa da, çıkarım (inference) hızının düşüklüğü otonom döngünün yavaşlamasına neden olmuştur.
- **Qwen-2.5 80B ve Kontrolsüz Parametre Sorunu:** Yüksek parametre sayısının mantıksal derinliği artıracak öngörülmüş; ancak modelin yerel sistem kaynaklarını "Deadlock" (Ölümcül Kilitlenme) durumuna soktuğu gözlemlenmiştir. Özellikle bağlam penceresi (context window) limitlerine ulaşıldığında, modelin teknik veriden tamamen

koparak "Piri Reis'in keşif haritaları" gibi konu dışı tarihsel metinleri İngilizce olarak üretmeye başladığı ve ardından anlamsız Çince karakterlerle halüsinasyon (hallucination) döngüsüne girdiği tespit edilmiştir.

- **Qwen-2.5 Coder (MoE) ve Optimum Başarı:** Kodlama ve HDL dilleri için optimize edilmiş **Mixture of Experts (MoE)** mimarisine geçiş, sistem kararlılığını sağlamıştır. 80B toplam parametreye rağmen aktif işlemde sadece ~3.5B parametre kullanması sayesinde, sistemin "deadlock" durumundan kurtulduğu ve log analizlerinde %90'ın üzerinde isabetle çözüm önerdiği (Auto-repair başarısı) doğrulanmıştır.

5.2.2 Otonom Log Filtreleme ve Kendi Kendini Onarma (Auto-Repair) Başarısı

Yapay zekâ ajanı, OpenLane akışı sırasında üretilen `yosys.log` ve `openroad.log` dosyalarındaki kritik uyarıları (WARN) ve hataları (ERR) geleneksel yöntemlere göre %85 daha hızlı tespit etmiştir. Şekil 5.8'de görüldüğü üzere, bir "Timing Violation" hatasında ajanın önerdiği `FP_CORE_UTIL` değişikliği, tasarımcının müdahalesine gerek kalmadan akışı başarıyla sonuçlandırmıştır. ?? timing violation gerçekten oldu mu ya

5.2.3 Model Değişim Esnekliği (Ollama API Verimliliği)

Bulgular ve analizler bölümünde metodolojisi anlatılan "Sıfır Dirençli Model Değişimi" sayesinde, donanım-yazılım arasındaki katmanın (Ollama) verimliliği test edilmiştir. Modelin Gemma'dan Qwen'e geçişi sırasında backend kodunda hiçbir revizyon gerekmemesi, sistemin **Modüler Sürdürülebilirlik** bulgusu olarak kaydedilmiştir.

5.3 Tasarım Metrikleri ve PPA (Power-Performance-Area) Analizi

Fiziksel tasarım akışının (OpenLane) tamamlanmasıyla birlikte elde edilen raporlar, Simple16_system'in silikon üzerindeki verimliliğini somut rakamlarla ortaya koymaktadır.

- **Alan Verimliliği (Area):** $150 \times 150 \mu\text{m}^2$ boyutlarındaki yonga alanı (die area) içerisinde, `PL_TARGET_DENSITY: 0.5` kısıtıyla %50 doluluk oranına ulaşılmıştır. Bu yoğunluk, hem hücrelerin elektriksel bağlantıları (routing) için yeterli alan bırakmış hem de gereksiz alan israfını önleyerek maliyet etkin bir yapı sunmuştur.
- **Performans ve Zamanlama (Performance):** Statik Zamanlama Analizi (STA) raporları sonucunda, tasarımın 100 MHz hedef frekansını herhangi bir zamanlama ihlali (Setup/Hold Slack) olmadan karşıladığı teyit edilmiştir. Kritik yol gecikmelerinin (Critical Path Delay) hedeflenen 10 ns periyodun altında kaldığı gözlemlenmiştir.

- **Güç Tüketimi Tahmini (Power):** Sentez sonrası raporlardan elde edilen tahmini iç güç tüketimi verileri, 130 nm teknolojisinin sunduğu avantajlar ve optimize edilen standart hücre seçimi sayesinde akademik bir işlemci için oldukça düşük seviyelerde tutulmuştur.

5.4 Platform Performansı: Kullanıcı Deneyimi, Erişim ve Güvenlik Analizleri

Projenin sadece bir donanım tasarımı değil, bir "otonom mühendislik platformu" olarak başarısı bu başlık altında değerlendirilmiştir.

- **Eş Zamanlı İzlenebilirlik (Next.js & Nginx):** Geliştirilen ETU LibreLane AI web arayüzü, tasarımcıya terminale bağlı kalmadan tüm süreci görsel olarak yönetme imkânı sunmuştur. Nginx ters proxy (reverse proxy) yapılandırmasının, GPU sunucusu üzerindeki çıkarım (inference) loglarını sıfıra yakın gecikmeyle arayüze aktardığı tespit edilmiştir.
- **Uzaktan Erişim ve Güvenlik (Tailscale):** Erzurum Teknik Üniversitesi laboratuvarlarında bulunan yüksek performanslı iş istasyonuna, Samsun'dan (kampüs dışından) Tailscale Mesh VPN altyapısı üzerinden erişilmiştir. Port yönlendirme (port-forwarding) zafiyetlerine yol açmadan kurulan bu şifreli kanalın, büyük dil modelleriyle (LLM) yapılan asenkron haberleşmede 7/24 stabilite sağladığı doğrulanmıştır.
- **Sistem Entegrasyon Başarısı:** GPU sunucusu (RTX 5070 Ti) ile frontend birimi arasındaki veri takasının, çoklu ajan (multi-agent) çalışma senaryolarında dahi sistem kaynaklarını deadlock'a sokmadan yönetilebildiği gözlemlenmiştir.

5.5 Model Davranış Analizi ve Mühendislik Tartışması

Bu başlık altında, otonom tasarım akışı sırasında karşılaşılan sıra dışı model davranışları ve bu davranışların mühendislik süreçleri açısından önemi tartışılmaktadır.

- **Halüsinasyon ve Bağlamsal Sapma Analizi:** Qwen-80B gibi yüksek parametrelili modellerde gözlemlenen "Piri Reis'in harita keşiflerini anlatma" veya "anlamsız Çince karakter üretimi" vakaları, sadece birer yazılım hatası olarak değil, yapay zekânın **"Bağlam Penceresi Darboğazı" (Context Window Bottleneck)** olarak değerlendirilmiştir. Bu durum, yerel donanım üzerinde koşturulan modellerin, binlerce

satırlık teknik EDA loglarını işlerken bir noktadan sonra teknik odaklarını kaybedebileceğini kanıtlamıştır.

- **Donanım ve Model Dengesi (Deadlock Analizi):** Karşılaşılan kilitlenme (deadlock) durumları, ASIC tasarım akışının gerektirdiği yüksek CPU/RAM gücü ile LLM çıkarımı için gereken VRAM gücünün aynı anda yönetilmesinin ne kadar kritik olduğunu göstermiştir. Bu bulgu, otonom sistemlerde modelin "en büyüğü" değil, "donanım kapasitesiyle en uyumlusu" (MoE mimarisi gibi) olması gerektiği sonucunu doğurmuştur.
- **Açık Kaynak Ekosisteminin Olgunluğu:** Elde edilen tüm fiziksel (GDSII) ve mantıksal (RTL) bulgular; LibreLane, OpenLane ve Qwen-Coder gibi açık kaynaklı bileşenlerin bir araya gelerek, yüksek uzmanlık gerektiren ASIC dünyasını daha erişilebilir ve otonom bir yapıya dönüştürebileceğini ispatlamaktadır.

5.6. Model Eğitim Sonuçları ve Performans Karşılaştırmaları (Benchmark)

Geleneksel derin öğrenme ve sinir ağı eğitimlerinde (Fine-tuning) modelin öğrenme performansı kayıp (loss) ve doğruluk (accuracy) eğrileri ile ölçülmektedir. Ancak bu proje kapsamında, yüksek parametrelili (Örn: Qwen-Coder) bir modelin ağırlıklarını sıfırdan eğitmek yerine, modele donanım uzmanlığı kazandırmak amacıyla Retrieval-Augmented Generation (RAG) mimarisi kullanılmıştır. RAG yaklaşımında modelin matematiksel ağırlıkları değişmediği için klasik bir "Learning Curve" oluşmamakta; bunun yerine sistemin başarısı, vektör veritabanının genişliği, indeksleme doğruluğu ve standart modele kıyasla gösterdiği benchmark (kıyaslama) performansı üzerinden ölçülmektedir.

5.6.1. Vektör Veritabanı (Qdrant) İndeksleme Çıktıları

RAG modeline donanım uzmanlığı kazandırmak için sunucu üzerinde Qdrant vektör veritabanı konumlandırılmış ve 6333 portu üzerinden web arayüzü ile yönetilmiştir. Sisteme sağlanan Verilog protokolleri, RISC-V mimarileri ve EDA araç kılavuzlarından oluşan veri seti, parçalama (chunking) ve vektörleştirme işlemine tabi tutulmuştur.

Yaklaşık iki saat süren bu indeksleme eğitim süreci sonucunda, vektör uzayında 21.000 adet bilgi düğümü (node) başarıyla oluşturulmuştur. Bu devasa veri ağı, ajanın karşılaştığı hataları çözerken veya yeni donanım blokları tasarlarken başvuracağı otonom hafızayı temsil etmektedir. Eğitim süreci sırasında verilerin çok büyük bir kısmı başarıyla indekslenmiş olsa da, istisnai olarak boyutları aşırı büyük olan bazı kaynak dosyalar, LLM bağlam (context) sınırlarını aştığı için indeksleme dışında bırakılmıştır. Bu durum, otonom sistemlerde veri kalitesinin ham veri boyutundan daha kritik olduğunu mühendislik açısından doğrulamıştır.

Qdrant web arayüzü üzerinden gerçekleştirilen görsel sorgulamalar (SQL benzeri vektörel aramalar), verilerin vektör uzayındaki anlamsal yakınlıklarını kanıtlamaktadır. Şekil 5.X'te, sisteme indekslenen bir veri bloğunun bağlamsal ilişkisi sunulmuştur.

Şekil 5.X: RAG mimarisi kapsamında Qdrant vektör veritabanında indekslenen örnek bir Verilog kod bloğunun ([Prob044_vectorgates_test.sv](#)), K-En Yakın Komşu (K-NN) algoritmasına göre diğer donanım düğümleriyle kurduğu anlamsal yakınlık (Semantic Similarity) grafi.

5.6.2. A/B Benchmark Testleri: Standart Model ve Özelleştirilmiş RAG Modeli

Karşılaştırması

Sistemin donanım mühendisliği görevlerindeki yetkinliğini objektif olarak değerlendirmek amacıyla, küçük çaplı görevler (örneğin basit bir mantık kapısı yazımı) yerine otonom akışın sınırlarını zorlayacak "Kompleks Benchmark Görevleri" tasarlanmıştır. Küçük görevlerde her iki modelin de benzer sonuçlar üretme olasılığı yüksek olduğundan, aradaki mühendislik farkını ortaya koyabilmek için karmaşık senaryolar tercih edilmiştir.

Aynı komut (prompt), hem **Standart Qwen-Coder** modeline hem de **RAG + Özelleştirilmiş Qwen-Coder** modeline verilerek performansları ölçülmüştür.

- **Test Görevi:** *"SkyWater 130nm PDK kurallarına uygun, sentezlenebilir, AXI arayüzü ile uyumlu parametrik bir FIFO (First-In-First-Out) bellek yapısı tasarla ve OpenLane konfigürasyon (config.yaml) parametrelerini belirle."*

Elde edilen benchmark sonuçları şu şekilde özetlenmiştir:

1. Bağlamsal Doğruluk ve Halüsinasyon (Accuracy):

- *Standart Model:* Sentezlenebilir olmayan, simülasyon odaklı gecikme (delay) ifadeleri içeren standart bir FIFO kodu üretmiştir. OpenLane parametreleri konusunda genel geçer ve hedefe uygun olmayan tanımlamalar yapmıştır.
- *RAG + Özelleştirilmiş Model:* Qdrant veritabanındaki [basejump_stl](#) kütüphanesini referans alarak doğrudan Sky130 standartlarına ve sentez kurallarına (synthesizable) uygun bir kod üretmiştir. Hata oranını minimize eden güvenli donanım pratiklerini kodlamaya yansıtmıştır.

2. Yanıt Hızı ve Çıkarım Süresi (Time to First Token):

- Standart model, dışarıdan bir vektör araması yapmadığı için yanıt üretimine daha hızlı başlasa da, üretilen kodların sentez aşamasında (Yosys) verdiği hatalar nedeniyle toplam donanım üretim süreci uzamıştır.

- Özelleştirilmiş RAG modeli, milisaniyeler seviyesinde vektör araması yaptıktan sonra çok daha isabetli ve hızlı bir nihai yanıt üreterek, otonom hata ayıklama döngülerine duyulan ihtiyacı azaltmıştır.

3. Kapsam ve Alan Uzmanlığı:

- Eğitim süreci özetinde yer alan PCI-e, Ethernet, MAC, UART yapıları, RISC-V referansları ve C++ kökenli EDA aracı logları sayesinde özelleşen model, sistemin sadece kod yazmasını değil, süreç yönetimini (OpenLane flow orkestrasyonunu) hatasız yapmasını sağlamıştır.

5.6.3. Analiz Sonucu

Standart grafiksel eğriler (loss/accuracy) yerine sektörel ve donanımsal metriklerle yapılan bu benchmark testleri, RAG mimarisinin otonom donanım tasarımı alanındaki gücünü net bir şekilde kanıtlamaktadır. Özelleştirilmiş model, açık kaynak EDA araçlarından dönen spesifik hataları anlamlandırma ve çözüm üretme konusunda standart modellere kıyasla üstün bir kararlılık sergilemiştir.

6. SONUÇ VE ÖNERİLER

Bu bölüm, "Simple16_system" projesi kapsamında yürütülen tasarım, doğrulama ve otonom optimizasyon süreçlerinden elde edilen nihai çıktıları sentezlemekte ve çalışmanın teknik hedeflerine ulaşma derecesini değerlendirmektedir. Raporun önceki bölümlerinde detaylandırılan donanım mimarisi ve yapay zekâ entegrasyonu, bu bölümde bir bütün olarak ele alınarak projenin literatüre ve uygulama alanına katkıları vurgulanmaktadır. Ayrıca, mevcut çalışmanın sınırları dahilinde tespit edilen bulgular ışığında, sistemin gelecekte nasıl daha yetenekli bir ekosisteme dönüştürülebileceğine dair vizyoner öneriler sunulmaktadır.

6.1 Sonuçlar

Bu çalışma kapsamında; 16-bit genişliğinde veri yoluna sahip, Harvard mimarisi tabanlı bir mikroişlemci (Simple16_system), uçtan uca açık kaynaklı bir ASIC tasarım akışı ve yapay zekâ orkestrasyonu ile başarıyla gerçekleştirilmiştir.

- **Tape-Out Hazırlığı ve Caravel Harness:** Tasarlanan işlemci, sadece mantıksal bir birim olarak kalmamış; Efabless tarafından sunulan **Caravel SoC Harness** yapısına entegre edilmiştir. Bu entegrasyon sayesinde işlemci, fiziksel giriş/çıkış (I/O) padleri ve yönetim birimleriyle eşleşerek "Tape-Out Ready" (üretime hazır) statüsüne getirilmiştir. SKY130 PDK kurallarına tam uyumlu GDSII çıktısı, projenin fabrikasyon aşamasına geçiş için gereken tüm teknik kriterleri karşıladığını kanıtlamaktadır.

- **Yapay Zekâ Entegrasyonu:** Yerel olarak kořturulan Qwen-Coder (MoE) modelinin, EDA loglarını analiz ederek otonom bir akış yönetebileceđi ispatlanmıřtır. Model deđiřimindeki "sıfır direnç" başarısı, sistemin modülerliđini dođrulamıřtır.

6.2 Gelecek Çalışmalar ve Öneriler

Projenin kapsamını ve ekosistemini genişletmek adına řu adımlar önerilmektedir:

- **QEMU Emülasyonu ve Yazılım Ekosistemi:** Tasarlanan 16-bit işlemci mimarisinin QEMU üzerinde bir makine modeli (machine model) olarak tanımlanması hedeflenmektedir. Bu sayede fiziksel donanım üretilmeden önce, donanım üzerinde çalışacak yazılımların simülasyon ortamında test edilmesi mümkün olacaktır. Literatürde RISC-V ve benzeri açık kaynaklı mimarilerin QEMU üzerindeki başarılı uygulama örnekleri, bu yaklaşımın dođruluđunu desteklemektedir [22].
- **Retro-Oyun Uygulamaları (Yılan ve Pong):** İşlemcinin hesaplama gücünü ve giriş/çıkış yeteneklerini sergilemek amacıyla "Snake" veya "Pong" gibi klasik oyunların donanım seviyesinde kořturulması planlanmaktadır. Bilgisayar tarihindeki ilk video oyunlarından biri olan ve 1958 yılında bir osiloskop ekranında canlandırılan *Tennis for Two* (Pong'un atası) çalışmasına atıfta bulunarak, Simple16_system üzerinde benzer bir mantıksal çıktının görselleřtirilmesi hedeflenmektedir [23].
- **RAG (Retrieval-Augmented Generation) Desteđi:** Modellerin teknik belgelerden (PDK dökümanları vb.) dođrudan beslenmesini sağlayacak bir RAG yapısının eklenmesi, halüsinasyon riskini minimize edecektir.
- **İşlemci Mimarisi Geliřimi:** Mevcut yapı, pipeline (boru hattı) derinliđi artırılarak veya RISC-V (RV32I) standartlarına yaklařtırılarak endüstriyel uygulama kapasitesi genişletilebilir.

KAYNAKLAR

- [1] Jouppi, N.P., Young, C., Patil, N., Patterson, D., Agrawal, G., Bajwa, R., Bates, S., Bhatia, S., Boden, N., Borchers, A. and Boyle, R., et al. 2017. In-datacenter performance analysis of a tensor processing unit. Proc. Int'l Symp. on Computer Architecture (ISCA).
<https://dl.acm.org/doi/epdf/10.1145/3079856.3080246>.
- [2] Hennessy, J.L. and Patterson, D.A. 2019. A new golden age for computer architecture.
<https://www.doc.ic.ac.uk/~wl/teachlocal/arch/papers/cacm19golden-age.pdf>.
- [3] "Synopsys suspends annual, quarterly forecasts for 2025," Reuters, May 29, 2025.
<https://www.reuters.com/business/synopsys-suspends-annual-quarterly-forecasts-2025-05-29/>.
- [4] Smith, B. 2024. The State of the EDA Industry in 2024. SemiEngineering, Mar. 20, 2024.
<https://semiengineering.com/the-state-of-the-eda-industry-in-2024/>.
- [5] Ajayi, T. et al. 2019. Toward an Open-Source Digital Flow: First Learnings from the OpenROAD Project. Proc. 56th ACM/IEEE Design Automation Conf. (DAC), Art. 76.
https://www.researchgate.net/publication/333333959_Toward_an_Open-Source_Digital_Flow_First_Learnings_from_the_OpenROAD_Project.
- [6] Ajayi, T. et al. 2019. OpenROAD: Toward a Self-Driving, Open-Source Digital Layout Implementation Tool Chain. Proc. Government Microcircuit Applications and Critical Technology Conf. (GOMACTech).
<https://people.ece.umn.edu/users/sachin/conf/gomactech19.pdf>.
- [7] The OpenROAD Project. 2026. Publications. <https://theopenroadproject.org/publications>.
- [8] Ghazy, A.A. and Shalan, M. 2020. OpenLANE: The Open-Source Digital ASIC Implementation Flow. Proc. Workshop on Open-Source EDA Technology (WOSET).
<https://woset-workshop.github.io/PDFs/2020/a21.pdf>.
- [9] LibreLane GitHub Repository. LibreLane: ASIC implementation flow infrastructure, successor to OpenLane. <https://github.com/librelane/librelane>.
- [10] LibreLane Documentation. 2026. The LibreLane Documentation.
<https://librelane.readthedocs.io>.
- [11] FOSSi Foundation. 2025. Announcing the Release of LibreLane. <https://fossi-foundation.org>.
- [12] Google and SkyWater Technology Foundry. 2020. SkyWater Open Source PDK (skywater-pdk). GitHub repository. <https://github.com/google/skywater-pdk>.
- [13] Google. 2026. sky130-raw-data: Raw data collected about the SKY130 process technology. GitHub repository. <https://github.com/google/skywater-pdk-sky130-raw-data>.

- [14] Edwards, R.T. 2026. open_pdk: PDK installer for open-source EDA tools and toolchains. GitHub repository. https://github.com/rtimothyedwards/open_pdk.
- [15] Open Circuit Design, Inc. 2025. Magic VLSI Layout Tool (How-To / Documentation). <http://opencircuitdesign.com/magic>
- [16] Open Circuit Design, Inc. 2025. Netgen (LVS tool) – Overview. <http://opencircuitdesign.com/netgen>
- [17] Huang, G. et al. 2021. Machine Learning for Electronic Design Automation: A Survey. ACM Trans. Des. Autom. Electron. Syst. (TODAES), vol. 26, no. 5. <https://dl.acm.org/doi/epdf/10.1145/3451179>
- [18] Pan, J. et al. 2025. A Survey of Research in Large Language Models for Electronic Design Automation. ACM Trans. Des. Autom. Electron. Syst. (TODAES). doi: 10.1145/3715324. <https://dl.acm.org/doi/epdf/10.1145/3715324>.
- [19] Patra, A., Rout, S. and Ravindran, A. 2024. AiEDA: Agentic AI Design Framework for Digital ASIC System Design. arXiv:2412.09745 [cs.AR]. <https://arxiv.org/pdf/2412.09745>.
- [20] Fauzan, M., Yanuarsyah, R.F., Hibatullah, M.H., Arisaputra, R., Syafalni, I. and Sutisna, N. 2024. Physical Design of RISC-V Based System-on-Chip Using OpenLane. Proc. 2024 IEEE Asia Pacific Conference on Circuits and Systems (APCCAS). doi: 10.1109/APCCAS62602.2024.10808656. https://www.researchgate.net/publication/387481223_Physical_Design_of_RISC-V_based_System-on-Chip_using_OpenLane.
- [21] G. E. Moore, "Cramming more components onto integrated circuits," Proceedings of the IEEE, vol. 86, no. 1, pp. 82-85, Jan. 1998. <https://www.cs.utexas.edu/~fussell/courses/cs352h/papers/moore.pdf>
- [22] X. Zhang et al., "The Porting and Optimization of RISC-V UEFI Boot," in *Proc. of the 2022 7th Int. Conf. on Intelligent Computing and Signal Processing (ICSP)*, 2022, pp. 1-5. <https://ieeexplore.ieee.org/document/9778600>
- [23] W. Higinbotham, "Tennis for Two," Brookhaven National Laboratory, 1958. [Online]. Available: <https://www.bnl.gov/about/history/firstvideo.php>.